Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Matplotlib Notes

10 Must-Know Graphs - Step by Step, Basic to Advanced

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225	+91 77220 82226	+91 77220 82227	+91 77220 82224
Pimpri-Chinchwad	Chh. Sambhajinagar	Nashik	Hadapsar, Pune

Matplotlib Notes

Data Visualization in Python -- 10 Must-Know Graphs, Step by Step

Prepared for Bikkad IT Institute Students | Level: Beginner to Advanced | Environment: Jupyter Notebook or Google Colab

Topic Index

0. Setup & Meet the Datasets
1. Line Plot -- plt.plot()
2. Scatter Plot -- plt.scatter()
3. Bar Chart -- plt.bar()
4. Grouped & Stacked Bar Chart
5. Histogram -- plt.hist()
6. Box Plot -- plt.boxplot()
7. Pie Chart -- plt.pie()
8. Subplots -- plt.subplot() / plt.subplots()

How to use these notes: every topic builds ONE chart from scratch, adding one or two Matplotlib functions per step. Each step shows the complete cell (so you can copy-paste and run it), the real chart it produces, and an explanation. **Lines printed in bold are the lines that are new or changed in that step** -- watch how each one changes the picture. Run the cells of a topic in order, from top to bottom.

0. Setup & Meet the Datasets

All examples use six small, everyday-life datasets (provided as CSV files and as one combined Excel workbook, one sheet each). Each one tells a story that students already know from daily life, so they can focus on learning Matplotlib instead of struggling to understand the data. First, import the three libraries:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Then read all six datasets **once** at the start of the notebook -- every example after this reuses these DataFrames:

```
steps = pd.read_csv('daily_steps_tracker.csv')
food = pd.read_csv('food_delivery_orders.csv')
poll = pd.read_csv('street_food_poll.csv')
expenses = pd.read_csv('monthly_household_expenses.csv')
marks = pd.read_csv('student_exam_marks.csv')
weather = pd.read_csv('pune_monthly_weather.csv')
```

Dataset 1: daily_steps_tracker.csv (variable: steps)

Two friends, Rahul (office job, loves weekend treks) and Sneha (walks to college daily), log their phone step-count for 2 weeks. Ordered days on the x-axis make it the perfect Line Plot dataset.

Shape: 14 rows x 4 columns | **Used in:** Line Plot, Styles | **First 5 rows:**

day	weekday	rahul_steps	sneha_steps
1	Mon	6200	9800
2	Tue	5800	10400
3	Wed	7100	9100
4	Thu	6500	11200
5	Fri	5400	10800

Dataset 2: food_delivery_orders.csv (variable: food)

120 food-app orders: cuisine, Veg/Non-Veg, distance from the restaurant, delivery time, bill and the customer's star rating. Does a longer distance mean a longer wait? Does a late order get a bad rating?

Shape: 120 rows x 7 columns | **Used in:** Scatter Plot, Histogram | **First 5 rows:**

order_id	cuisine	food_type	distance_km	delivery_time_min	bill_amount	rating
ORD1001	South Indian	Veg	8.4	37	120	4.4
ORD1002	Pizza	Non-Veg	1.5	11	400	5.0
ORD1003	Chinese	Veg	1.5	10	300	5.0
ORD1004	South Indian	Veg	1.2	12	170	4.8
ORD1005	Biryani	Non-Veg	6.1	31	450	4.8

Dataset 3: street_food_poll.csv (variable: poll)

A class poll: 'Which street food is your favourite?' -- votes from boys and girls for Vada Pav, Pani Puri, Misal Pav and more.

Shape: 7 rows x 3 columns | **Used in:** Bar, Horizontal Bar, Grouped & Stacked Bar | **First 5 rows:**

food	boys_votes	girls_votes
Vada Pav	34	22
Pani Puri	18	41
Misal Pav	27	19
Pav Bhaji	25	24
Momos	15	29

Dataset 4: monthly_household_expenses.csv (variable: expenses)

One Pune family's monthly income and spending (rent, groceries, electricity, travel, entertainment, shopping) for a full year -- watch the summer AC bill and the Diwali shopping spike!

Shape: 12 rows x 8 columns | **Used in:** Pie Chart, Area / Stack Plot | **First 5 rows:**

month	income	rent	groceries	electricity	travel	entertainment	shopping
Jan	60000	16000	10500	1800	4500	3200	4000
Feb	60000	16000	10200	1900	4200	2900	3500
Mar	60000	16000	11000	2600	4800	3500	4200
Apr	60000	16000	10800	3900	5200	3800	5000
May	60000	16000	11500	4400	20000	7200	6500

Dataset 5: student_exam_marks.csv (variable: marks)

Marks of 30 students in Maths, Science, English and Computer, including a few surprising very low scores (outliers).

Shape: 30 rows x 6 columns | **Used in:** Box Plot | **First 5 rows:**

roll_no	name	maths	science	english	computer
101	Aarav	56	69	72	75
102	Ishita	50	61	70	79
103	Rohan	48	76	78	95
104	Sneha	57	55	68	38
105	Kunal	85	64	67	83

Dataset 6: pune_monthly_weather.csv (variable: weather)

Typical (approximate) Pune climate month by month: max/min temperature, rainfall, rainy days and humidity -- hot April-May, heavy monsoon July.

Shape: 12 rows x 6 columns | **Used in:** Subplots, twinx() | **First 5 rows:**

month	max_temp_c	min_temp_c	rainfall_mm	rainy_days	humidity_pct
Jan	30	12	1	0	45
Feb	32	13	0	0	38
Mar	35	17	3	0	32
Apr	38	21	10	1	35
May	37	23	30	3	48

Teaching Note

Before drawing anything, let students run steps.head(), food.info() and marks.describe() and say in their own words what each dataset is about. Two minutes spent understanding the data makes every chart that follows feel obvious.

1. Line Plot -- plt.plot()

A line plot joins data points with straight lines. It is a **bivariate** chart (one x sequence + one y sequence) and is the natural choice when the x-axis is **ordered** -- days, months, years. Use case: tracking how a number changes over time (a trend). Here we compare how many steps Rahul and Sneha walk every day for two weeks.

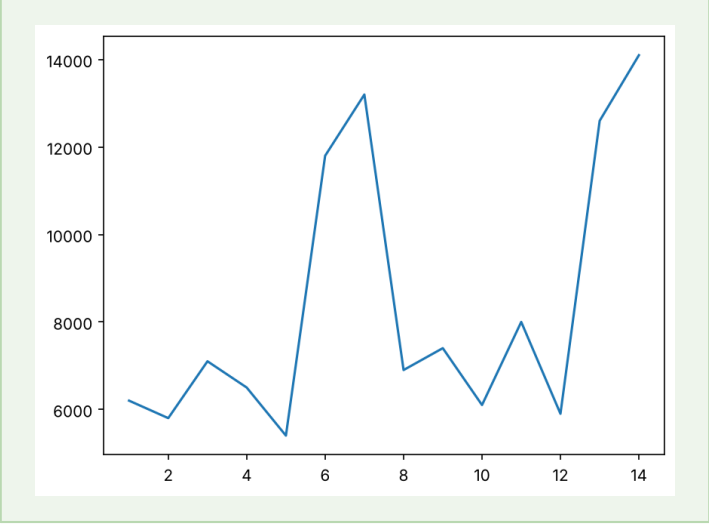
Dataset: daily_steps_tracker.csv

Syntax

```
plt.plot(x, y, color=, linestyle=, linewidth=, marker=,
         markersize=, label=)
plt.title()  plt.xlabel()  plt.ylabel()  plt.legend(loc=)
plt.figure(figsize=(w, h))  plt.xticks()  plt.grid()
plt.axhline(y)  plt.annotate()  plt.ylim()  plt.show()
```

1.1 The plainest possible line plot

```
plt.plot(steps['day'], steps['rahul_steps'])
```

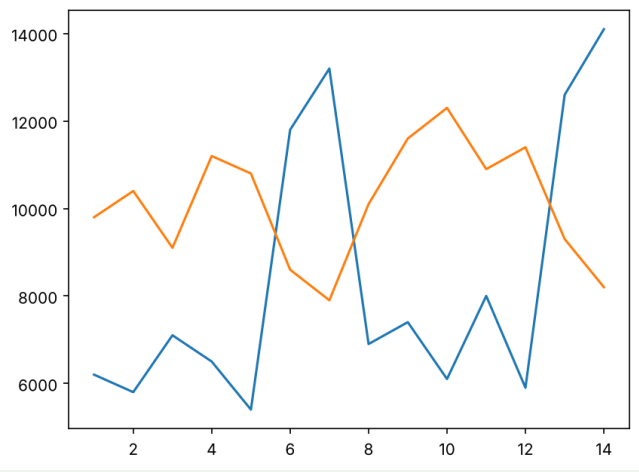


Explanation

plt.plot(x, y) is all that is needed. Matplotlib takes day numbers 1-14 for the x-axis and Rahul's steps for the y-axis, and joins the points in order with one default blue line. No title, no labels -- just the raw trend. Notice the two big jumps: those are Rahul's weekends.

1.2 Adding a second line on the same axes

```
plt.plot(steps['day'], steps['rahul_steps'])
plt.plot(steps['day'], steps['sneha_steps'])
```

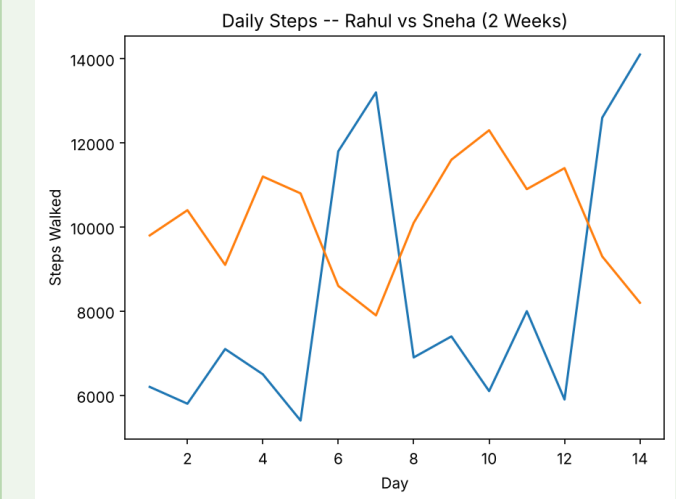


Explanation

Every plt.plot() call draws on the same current figure, so a second call simply adds a second line. Matplotlib automatically picks a new colour (orange) so the two lines stay distinguishable. Sneha is steady on weekdays but walks less on weekends -- the exact opposite of Rahul.

1.3 title(), xlabel() and ylabel()

```
plt.plot(steps['day'], steps['rahul_steps'])
plt.plot(steps['day'], steps['sneha_steps'])
plt.title('Daily Steps -- Rahul vs Sneha (2 Weeks)')
plt.xlabel('Day')
plt.ylabel('Steps Walked')
```

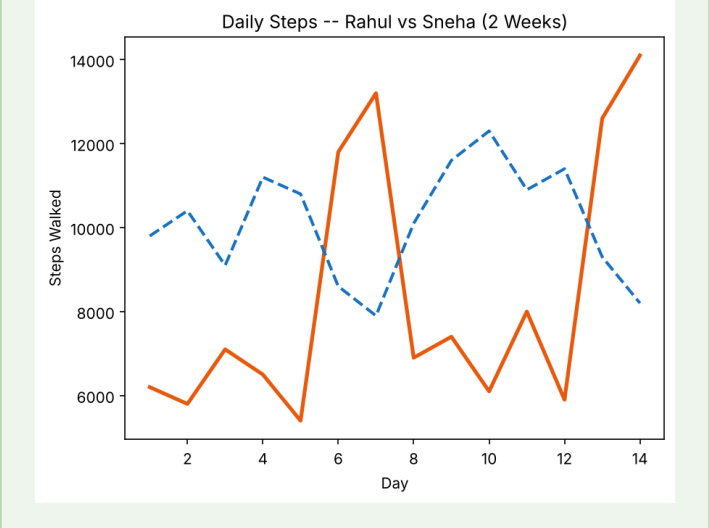


Explanation

plt.title() writes a heading above the chart; plt.xlabel() and plt.ylabel() caption the two axes. A chart without labels forces the reader to guess what the numbers mean -- always add these three.

1.4 color=, linestyle= and linewidth=

```
plt.plot(steps['day'], steps['rahul_steps'], color='#E8590C',
         linestyle='solid', linewidth=2.5)
plt.plot(steps['day'], steps['sneha_steps'], color='#1971C2',
         linestyle='dashed', linewidth=2)
plt.title('Daily Steps -- Rahul vs Sneha (2 Weeks)')
plt.xlabel('Day')
plt.ylabel('Steps Walked')
```

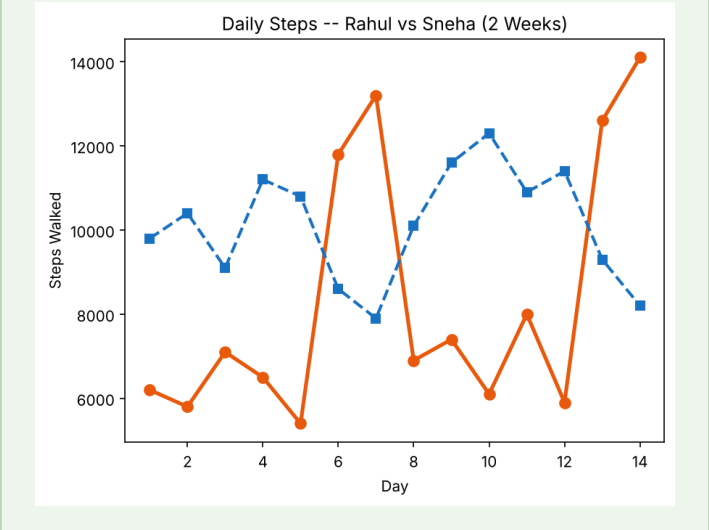


Explanation

color= accepts a name ('red'), a short code ('r') or a hex code ('#E8590C'). linestyle= can be 'solid', 'dashed', 'dotted' or 'dashdot', and linewidth= sets the thickness. A dashed line keeps Sneha's line readable even on a black-and-white print.

1.5 marker= and markersize=

```
plt.plot(steps['day'], steps['rahul_steps'], color='#E8590C',
         linestyle='solid', linewidth=2.5,
         marker='o', markersize=7)
plt.plot(steps['day'], steps['sneha_steps'], color='#1971C2',
         linestyle='dashed', linewidth=2,
         marker='s', markersize=6)
plt.title('Daily Steps -- Rahul vs Sneha (2 Weeks)')
plt.xlabel('Day')
plt.ylabel('Steps Walked')
```

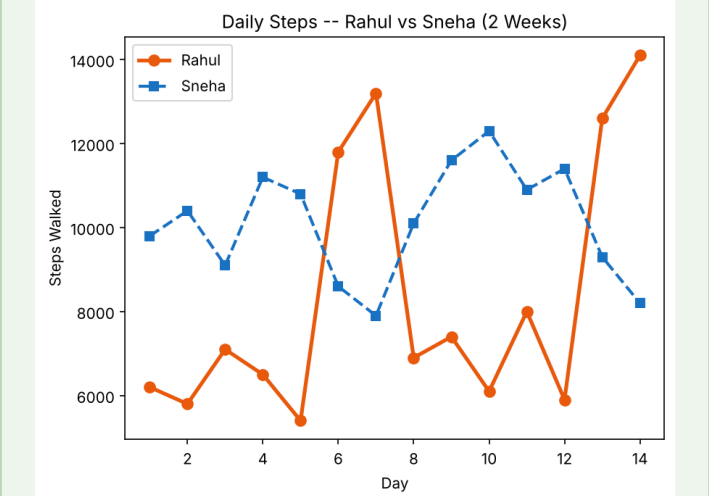


Explanation

marker= stamps a shape on every real data point -- 'o' circle, 's' square, 'D' diamond, '^' triangle, '*' star. markersize= controls how big it is. Now each of the 14 actual readings is clearly visible, not just the line between them.

1.6 label= and plt.legend()

```
plt.plot(steps['day'], steps['rahul_steps'], color='#E8590C',
         linestyle='solid', linewidth=2.5,
         marker='o', markersize=7, label='Rahul')
plt.plot(steps['day'], steps['sneha_steps'], color='#1971C2',
         linestyle='dashed', linewidth=2,
         marker='s', markersize=6, label='Sneha')
plt.title('Daily Steps -- Rahul vs Sneha (2 Weeks)')
plt.xlabel('Day')
plt.ylabel('Steps Walked')
plt.legend(loc='upper left')
```

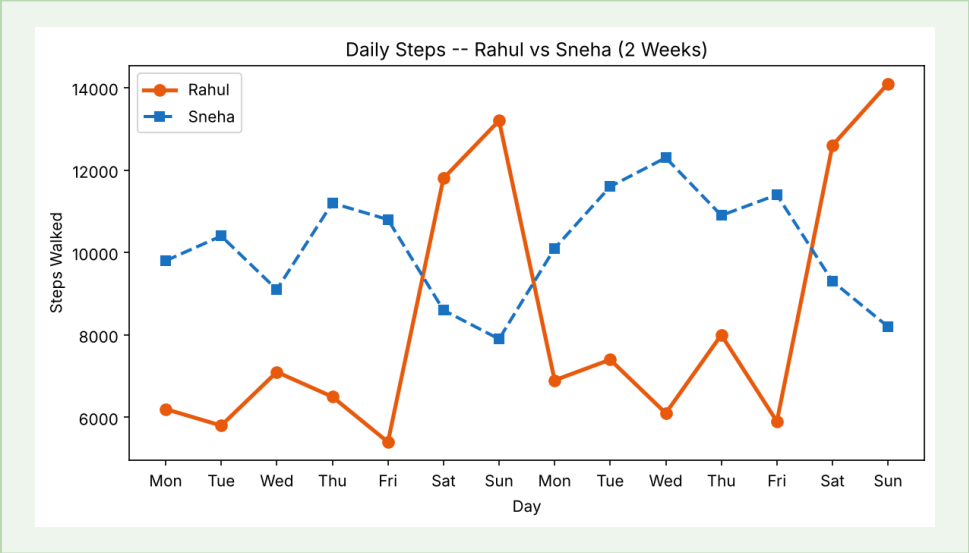


Explanation

label= gives each line a name, but nothing appears until plt.legend() is called -- it collects every label= and draws a key box. loc= decides where: 'upper left', 'lower right', 'center', or 'best' (Matplotlib picks the emptiest corner).

1.7 plt.figure(figsize=) and plt.xticks()

```
plt.figure(figsize=(9, 4.5))
plt.plot(steps['day'], steps['rahul_steps'], color='#E8590C',
         linestyle='solid', linewidth=2.5,
         marker='o', markersize=7, label='Rahul')
plt.plot(steps['day'], steps['sneha_steps'], color='#1971C2',
         linestyle='dashed', linewidth=2,
         marker='s', markersize=6, label='Sneha')
plt.title('Daily Steps -- Rahul vs Sneha (2 Weeks)')
plt.xlabel('Day')
plt.ylabel('Steps Walked')
plt.legend(loc='upper left')
plt.xticks(steps['day'], steps['weekday'])
```

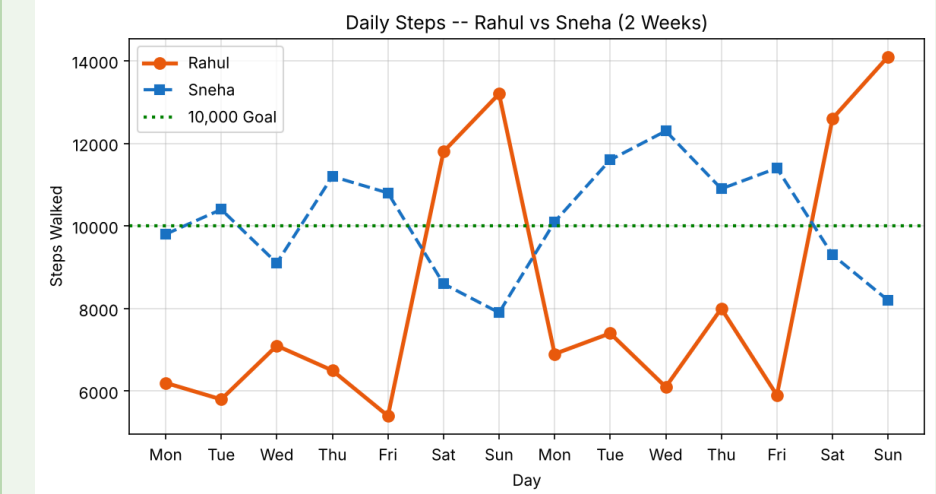


Explanation

plt.figure(figsize=(width, height)) must come FIRST -- it creates a new, wider canvas measured in inches. plt.xticks(positions, labels) replaces the numbers 1-14 with the weekday names, so now everyone can see that Rahul's peaks fall on Sat and Sun.

1.8 plt.grid() and plt.axhline() -- a goal line

```
plt.figure(figsize=(9, 4.5))
plt.plot(steps['day'], steps['rahul_steps'], color='#E8590C',
         linestyle='solid', linewidth=2.5,
         marker='o', markersize=7, label='Rahul')
plt.plot(steps['day'], steps['sneha_steps'], color='#1971C2',
         linestyle='dashed', linewidth=2,
         marker='s', markersize=6, label='Sneha')
plt.axhline(10000, color='green', linestyle=':', linewidth=2,
            label='10,000 Goal')
plt.title('Daily Steps -- Rahul vs Sneha (2 Weeks)')
plt.xlabel('Day')
plt.ylabel('Steps Walked')
plt.legend(loc='upper left')
plt.xticks(steps['day'], steps['weekday'])
plt.grid(alpha=0.4)
```

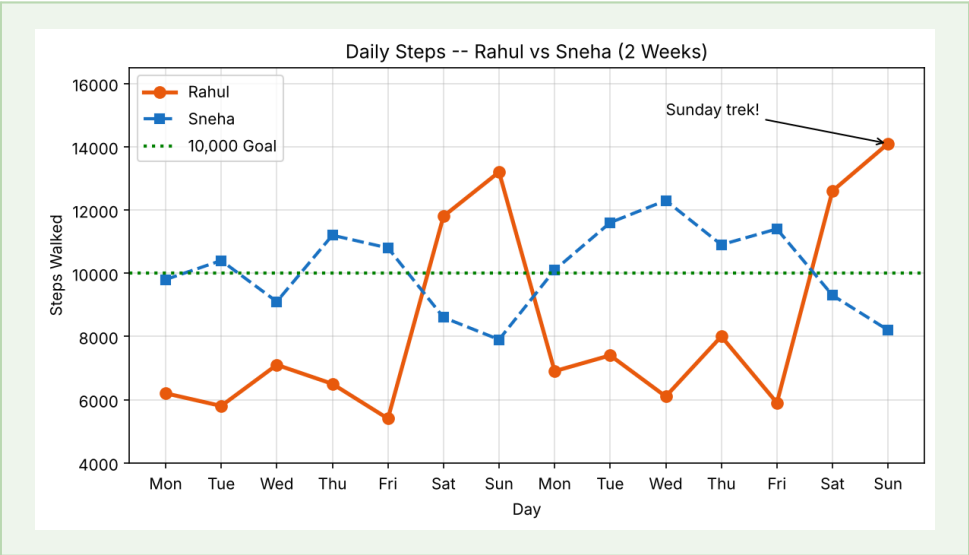


Explanation

plt.axhline(y) draws a horizontal reference line across the whole chart -- here the popular 10,000-steps goal. Because it has a label= and comes before plt.legend(), it also appears in the legend. plt.grid(alpha=0.4) adds faint gridlines so values are easy to read off.

1.9 plt.annotate() -- pointing at an interesting point

```
plt.figure(figsize=(9, 4.5))
plt.plot(steps['day'], steps['rahul_steps'], color='#E8590C',
         linestyle='solid', linewidth=2.5,
         marker='o', markersize=7, label='Rahul')
plt.plot(steps['day'], steps['sneha_steps'], color='#1971C2',
         linestyle='dashed', linewidth=2,
         marker='s', markersize=6, label='Sneha')
plt.axhline(10000, color='green', linestyle=':', linewidth=2,
            label='10,000 Goal')
plt.annotate('Sunday trek!', xy=(14, 14100), xytext=(10, 15000),
             arrowprops=dict(arrowstyle='->', color='black'))
plt.title('Daily Steps -- Rahul vs Sneha (2 Weeks)')
plt.xlabel('Day')
plt.ylabel('Steps Walked')
plt.legend(loc='upper left')
plt.xticks(steps['day'], steps['weekday'])
plt.grid(alpha=0.4)
plt.ylim(4000, 16500)
```

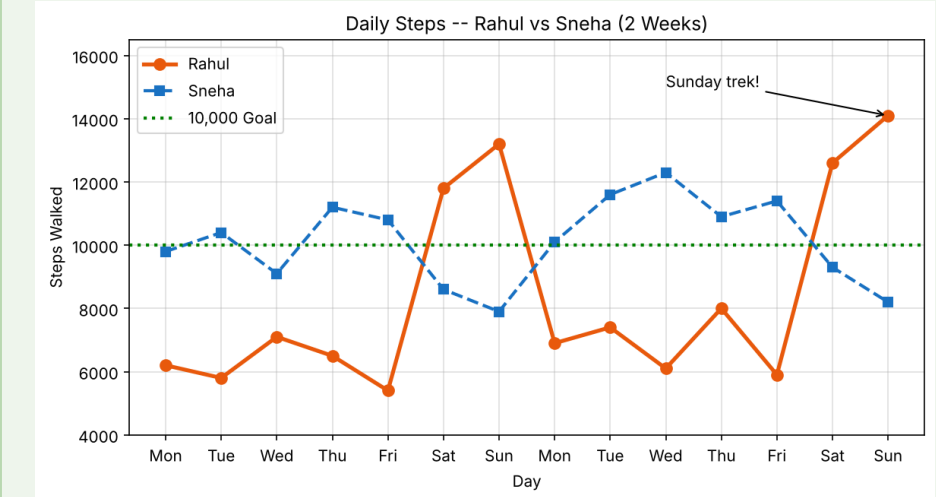


Explanation

plt.annotate(text, xy=, xytext=, arrowprops=) writes text at xytext and draws an arrow to the data point at xy -- great for telling the story of a chart. plt.ylim(4000, 16500) widens the visible y-range so the note has room at the top.

1.10 plt.show() -- the finished chart

```
plt.figure(figsize=(9, 4.5))
plt.plot(steps['day'], steps['rahul_steps'], color='#E8590C',
         linestyle='solid', linewidth=2.5,
         marker='o', markersize=7, label='Rahul')
plt.plot(steps['day'], steps['sneha_steps'], color='#1971C2',
         linestyle='dashed', linewidth=2,
         marker='s', markersize=6, label='Sneha')
plt.axhline(10000, color='green', linestyle=':', linewidth=2,
            label='10,000 Goal')
plt.annotate('Sunday trek!', xy=(14, 14100), xytext=(10, 15000),
             arrowprops=dict(arrowstyle='->', color='black'))
plt.title('Daily Steps -- Rahul vs Sneha (2 Weeks)')
plt.xlabel('Day')
plt.ylabel('Steps Walked')
plt.legend(loc='upper left')
plt.xticks(steps['day'], steps['weekday'])
plt.grid(alpha=0.4)
plt.ylim(4000, 16500)
plt.show()
```



Explanation

plt.show() renders everything built above as one finished chart (in Jupyter/Colab it appears below the cell; in a .py script it opens a window). Starting from one bare line, ten small steps produced a clear, labelled, story-telling chart.

Teaching Note

Ask students to change only ONE argument at a time -- e.g. marker='o' to marker='*', or loc='upper left' to loc='best' -- and re-run. Seeing a single change is the fastest way to remember what each argument does.

2. Scatter Plot -- plt.scatter()

A scatter plot draws one independent dot for every (x, y) pair -- no connecting line. It is a **numerical vs numerical** chart, and its classic use case is checking whether two numbers are **related (correlated)**. Here we ask: does a restaurant that is farther away take longer to deliver?

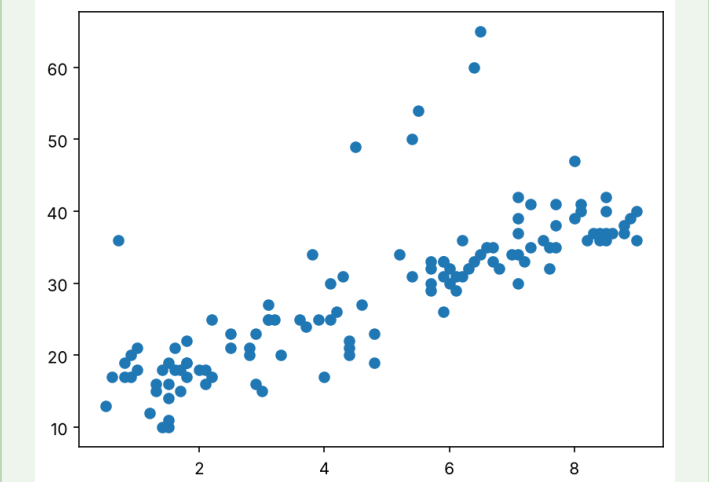
Dataset: food_delivery_orders.csv

Syntax

```
plt.scatter(x, y, color=, marker=, s=, alpha=,
            edgecolor=, c=, cmap=, label=)
plt.colorbar(label=)
np.polyfit(x, y, 1)   # slope & intercept of a trend line
```

2.1 A plain scatter plot

```
plt.scatter(food['distance_km'], food['delivery_time_min'])
```

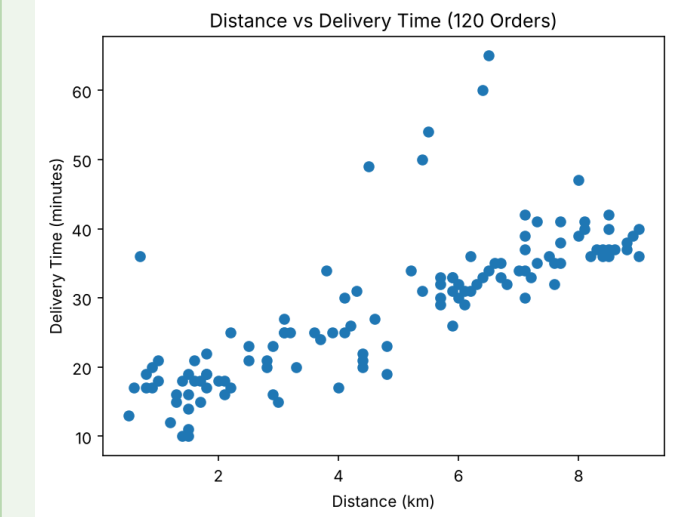


Explanation

Each of the 120 orders becomes one dot: distance on the x-axis, delivery time on the y-axis. The cloud clearly rises from left to right -- longer distance, longer wait. That upward drift is a **positive correlation**.

2.2 Title and axis labels

```
plt.scatter(food['distance_km'], food['delivery_time_min'])
plt.title('Distance vs Delivery Time (120 Orders)')
plt.xlabel('Distance (km)')
plt.ylabel('Delivery Time (minutes)')
```

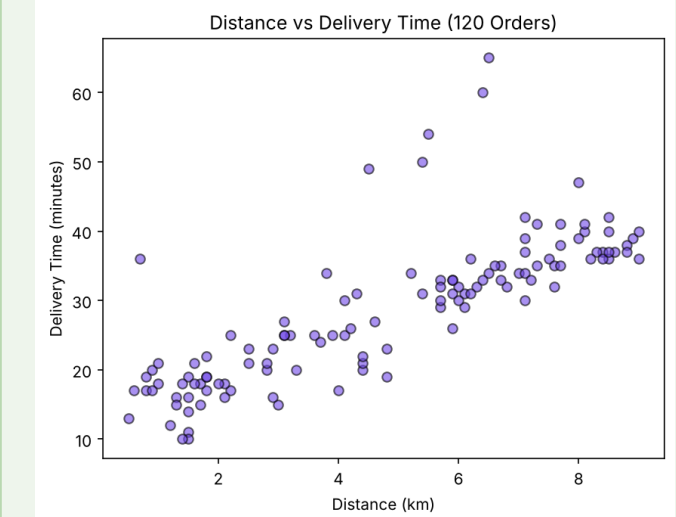


Explanation

Exactly the same labelling functions as the line plot. Notice a few dots sitting far above the rest -- those are orders that were delivered very late (rainy evenings).

2.3 color=, marker=, alpha= and edgecolor=

```
plt.scatter(food['distance_km'], food['delivery_time_min'],
            color='#7048E8', marker='o', alpha=0.6,
            edgecolor='black')
plt.title('Distance vs Delivery Time (120 Orders)')
plt.xlabel('Distance (km)')
plt.ylabel('Delivery Time (minutes)')
```

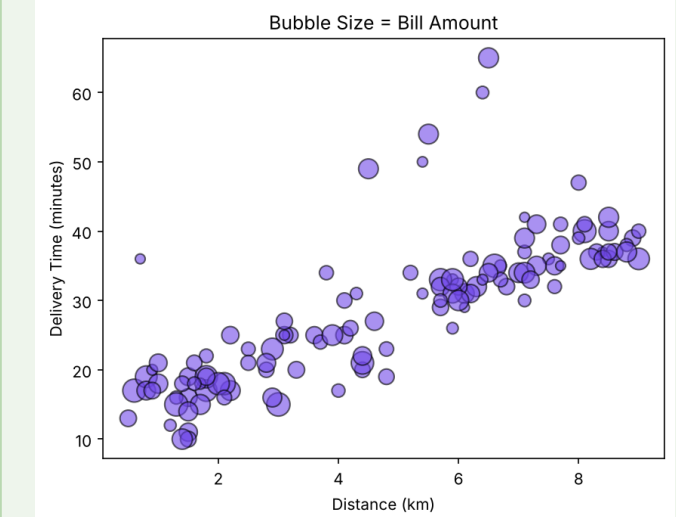


Explanation

alpha= sets transparency from 0 (invisible) to 1 (solid). With alpha=0.6, overlapping dots look darker, so crowded areas stand out. edgecolor= draws a thin outline around every dot.

2.4 s= -- bubble size from a third column

```
plt.scatter(food['distance_km'], food['delivery_time_min'],
            color='#7048E8', marker='o', alpha=0.6,
            edgecolor='black', s=food['bill_amount'] / 3)
plt.title('Bubble Size = Bill Amount')
plt.xlabel('Distance (km)')
plt.ylabel('Delivery Time (minutes)')
```

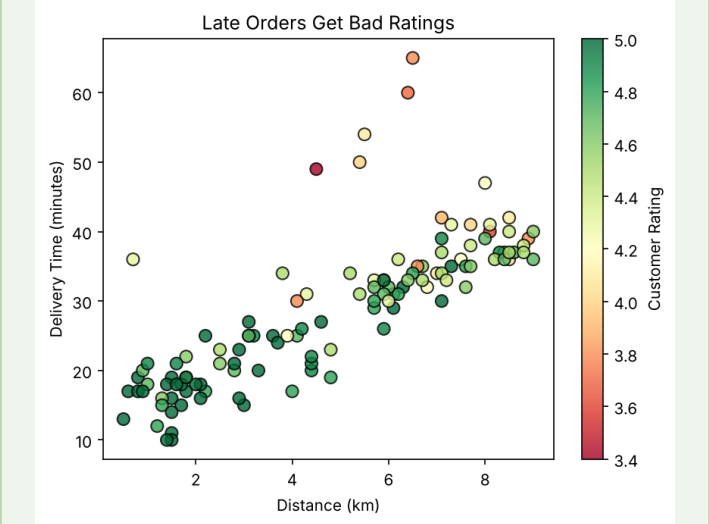


Explanation

s= is the marker area and it accepts a whole column. Passing bill_amount (divided by 3 so bubbles are not too big) turns a 2-variable scatter into a 3-variable **bubble chart** -- bigger bubble, bigger bill.

2.5 c= and cmap= -- colour from a number, plus colorbar()

```
plt.scatter(food['distance_km'], food['delivery_time_min'],
            c=food['rating'], cmap='RdYlGn', alpha=0.8,
            edgecolor='black', s=60)
plt.colorbar(label='Customer Rating')
plt.title('Late Orders Get Bad Ratings')
plt.xlabel('Distance (km)')
plt.ylabel('Delivery Time (minutes)')
```

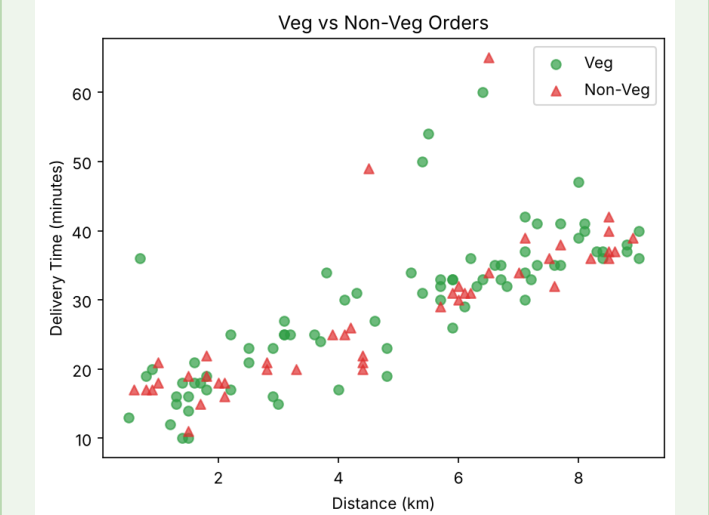


Explanation

c= colours every dot by a numeric column, and cmap= chooses the colour scale -- 'RdYlGn' runs Red (low) to Green (high). plt.colorbar() adds the scale on the right. Now the story pops out: the red dots (low ratings) are the late deliveries at the top.

2.6 One scatter() per group + legend

```
veg = food[food['food_type'] == 'Veg']
nonveg = food[food['food_type'] == 'Non-Veg']
plt.scatter(veg['distance_km'], veg['delivery_time_min'],
            color='#2F9E44', alpha=0.7, label='Veg')
plt.scatter(nonveg['distance_km'], nonveg['delivery_time_min'],
            color='#E03131', alpha=0.7, marker='^', label='Non-Veg')
plt.title('Veg vs Non-Veg Orders')
plt.xlabel('Distance (km)')
plt.ylabel('Delivery Time (minutes)')
plt.legend()
```

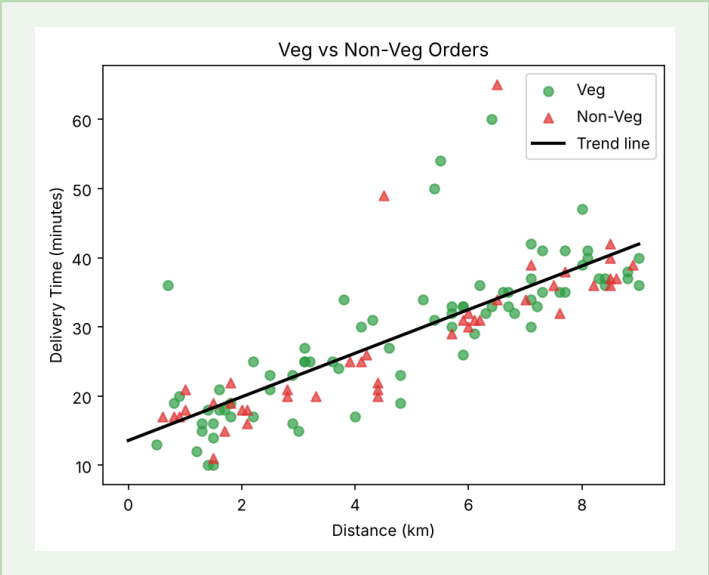


Explanation

To colour by a **category** (text) instead of a number, filter the DataFrame into groups and call plt.scatter() once per group with its own color=, marker= and label=. plt.legend() then explains which colour is which.

2.7 Adding a trend line with np.polyfit()

```
veg = food[food['food_type'] == 'Veg']
nonveg = food[food['food_type'] == 'Non-Veg']
plt.scatter(veg['distance_km'], veg['delivery_time_min'],
            color='#2F9E44', alpha=0.7, label='Veg')
plt.scatter(nonveg['distance_km'], nonveg['delivery_time_min'],
            color='#E03131', alpha=0.7, marker='^', label='Non-Veg')
slope, intercept = np.polyfit(food['distance_km'],
                              food['delivery_time_min'], 1)
x_line = np.linspace(0, 9, 50)
y_line = slope * x_line + intercept
plt.plot(x_line, y_line, color='black', linewidth=2,
         label='Trend line')
plt.title('Veg vs Non-Veg Orders')
plt.xlabel('Distance (km)')
plt.ylabel('Delivery Time (minutes)')
plt.legend()
plt.show()
```



Explanation

np.polyfit(x, y, 1) finds the best straight line through the dots and returns its slope and intercept (here roughly 3 extra minutes per km). We build 50 x values with np.linspace(), compute y = slope * x + intercept, and draw them with plt.plot() -- a scatter and a line on the same axes.

Teaching Note

A scatter plot answers 'are these two numbers related?'; a line plot answers 'how did one number change over an ordered sequence?'. Ask students why plt.plot() on this food data (unsorted distances) would draw a meaningless zig-zag.

3. Bar Chart -- plt.bar()

A bar chart compares a number across separate **categories** using the height of rectangles. It is a **categorical vs numerical** chart. Use case: comparing totals or counts -- sales per store, votes per option, marks per subject. Our question: which street food won the class poll?

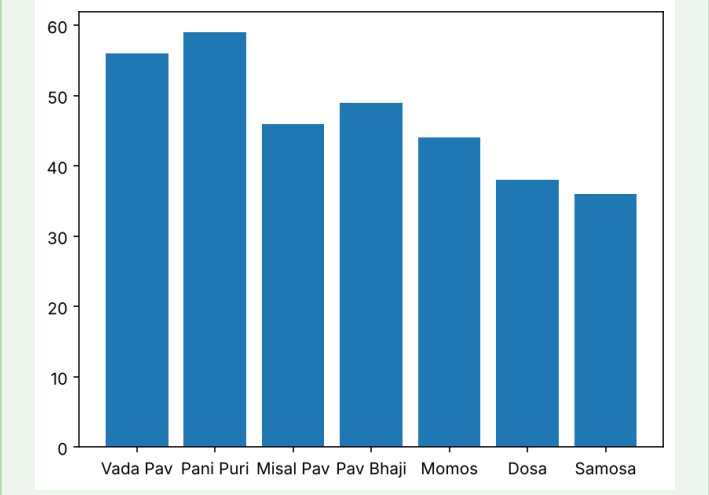
Dataset: street_food_poll.csv

Syntax

```
plt.bar(x, height, width=, color=, edgecolor=)
bars = plt.bar(...)   plt.bar_label(bars, padding=)
plt.xticks(rotation=)   df.sort_values(col, ascending=)
```

3.1 A plain vertical bar chart

```
poll['total_votes'] = poll['boys_votes'] + poll['girls_votes']
plt.bar(poll['food'], poll['total_votes'])
```

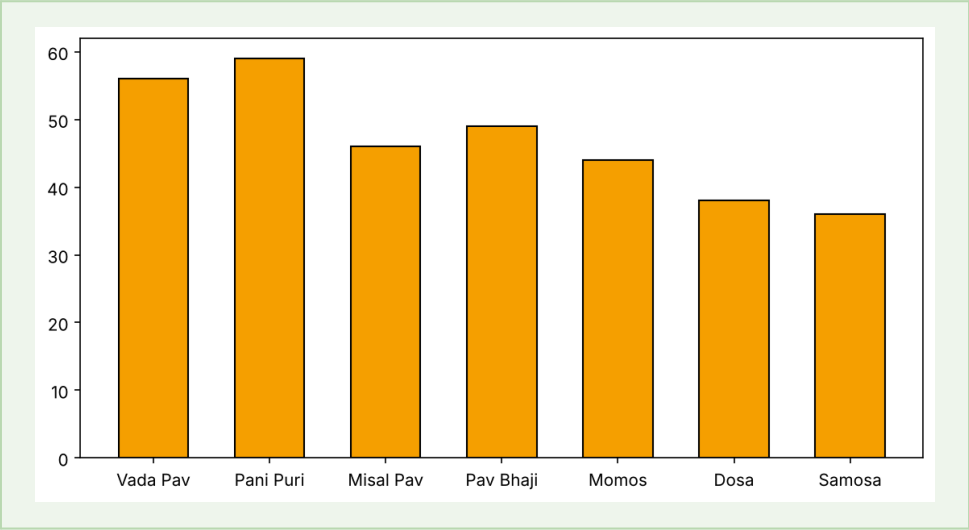


Explanation

First we add a total_votes column (boys + girls). plt.bar(x, height) then draws one bar per food, its height taken from total_votes. The food names become the x-axis labels automatically -- but on the small default canvas they look cramped.

3.2 figsize=, color=, edgecolor= and width=

```
plt.figure(figsize=(9, 4.5))
plt.bar(poll['food'], poll['total_votes'], color='#F59F00',
        edgecolor='black', width=0.6)
```

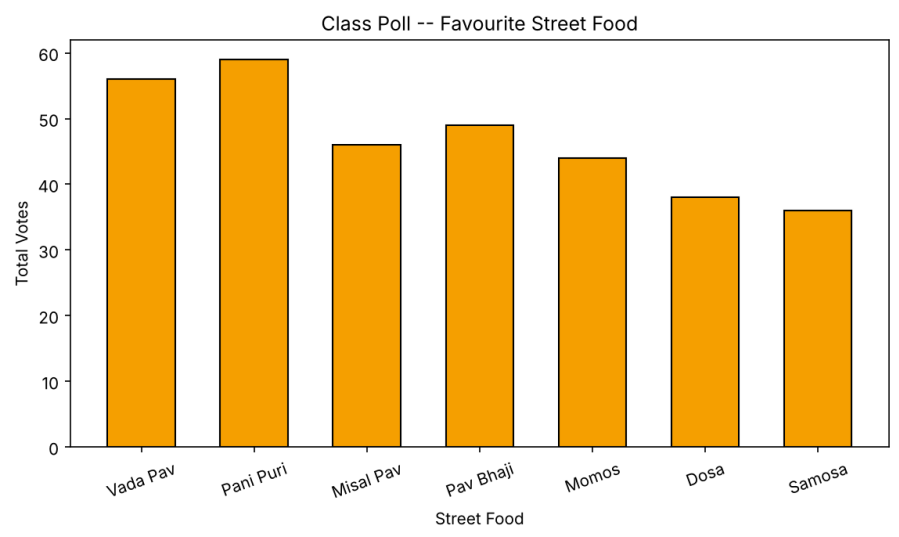


Explanation

A wider figure gives every label room. color= fills the bars, edgecolor= outlines them and width= (default 0.8) makes the bars slimmer so there is more white space between them.

3.3 Title, labels and plt.xticks(rotation=)

```
plt.figure(figsize=(9, 4.5))
plt.bar(poll['food'], poll['total_votes'], color='#F59F00',
        edgecolor='black', width=0.6)
plt.title("Class Poll -- Favourite Street Food")
plt.xlabel('Street Food')
plt.ylabel('Total Votes')
plt.xticks(rotation=20)
```

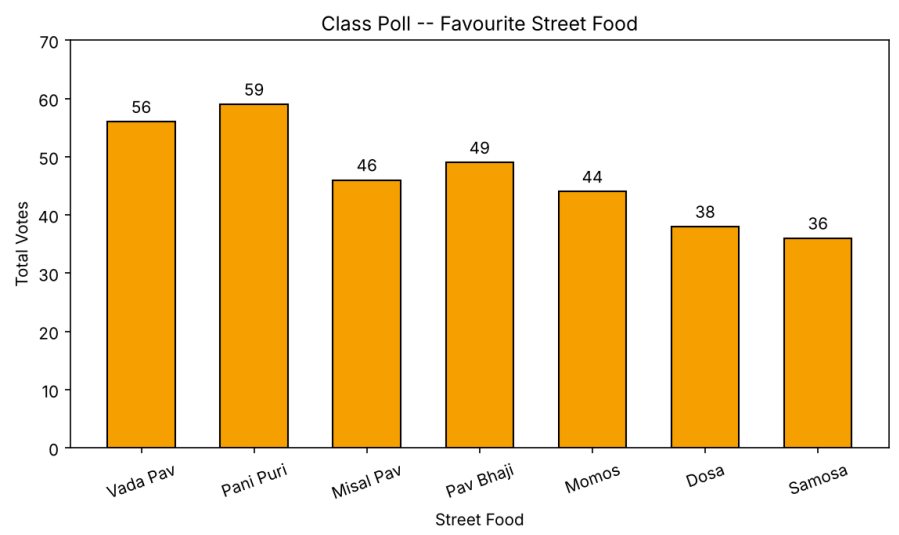


Explanation

plt.xticks(rotation=20) tilts the category labels by 20 degrees -- the standard fix whenever bar labels are long. Without it, names like 'Pani Puri' and 'Misal Pav' can run into each other on smaller screens.

3.4 plt.bar_label() -- values on top of bars

```
plt.figure(figsize=(9, 4.5))
bars = plt.bar(poll['food'], poll['total_votes'], color='#F59F00',
               edgecolor='black', width=0.6)
plt.bar_label(bars, padding=3)
plt.title("Class Poll -- Favourite Street Food")
plt.xlabel('Street Food')
plt.ylabel('Total Votes')
plt.xticks(rotation=20)
plt.ylim(0, 70)
```

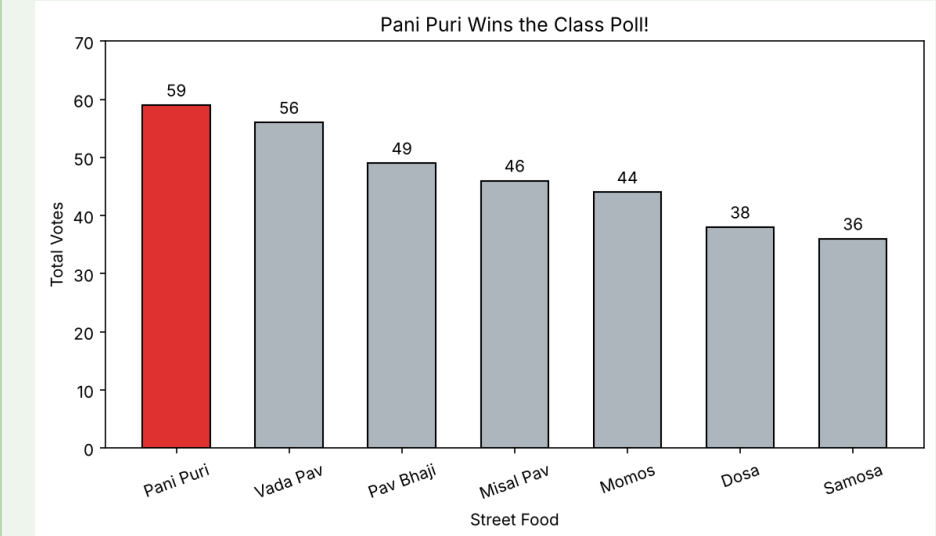


Explanation

plt.bar() returns the group of bars, which we store in bars. plt.bar_label(bars) writes each bar's exact value above it -- no more guessing from the axis. padding=3 lifts the number slightly; ylim(0, 70) makes room for the labels.

3.5 Sorting the bars and highlighting the winner

```
ranked = poll.sort_values('total_votes', ascending=False)
colours = ['#E03131'] + ['#ADB5BD'] * 6
plt.figure(figsize=(9, 4.5))
bars = plt.bar(ranked['food'], ranked['total_votes'], color=colours,
               edgecolor='black', width=0.6)
plt.bar_label(bars, padding=3)
plt.title('Pani Puri Wins the Class Poll!')
plt.xlabel('Street Food')
plt.ylabel('Total Votes')
plt.xticks(rotation=20)
plt.ylim(0, 70)
plt.show()
```



Explanation

sort_values() orders the foods from most to fewest votes, so the chart reads like a ranking. color= also accepts a list -- one colour per bar -- so the winner is red and the rest are grey. Highlighting just one bar tells the reader exactly where to look.

Teaching Note

Sorted bars are almost always easier to read than bars in random order. Ask students: which food is most popular with boys only? (Change total_votes to boys_votes and re-run.)

4. Grouped & Stacked Bar Chart

When each category has **two or more sub-groups** (boys and girls), a simple bar chart is not enough. A **grouped** bar chart puts the sub-group bars side by side -- best for comparing boys vs girls. A **stacked** bar chart piles them on top of each other -- best for seeing the total AND its make-up together.

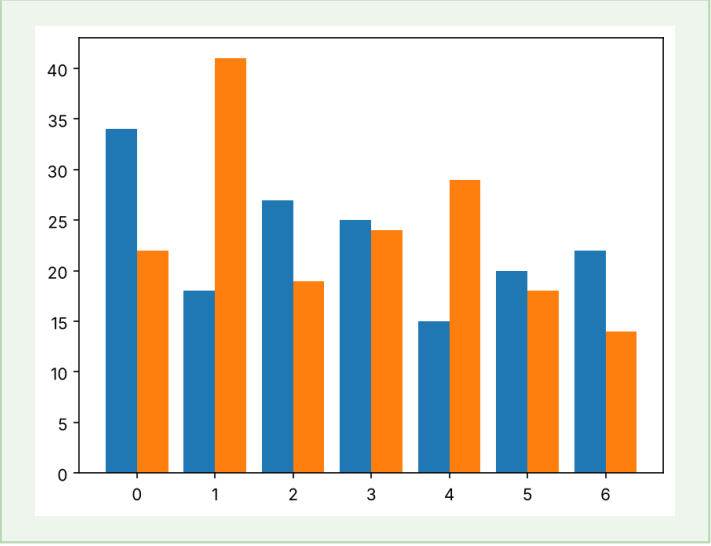
Dataset: street_food_poll.csv

Syntax

```
x = np.arange(n)   # numeric bar positions 0, 1, 2 ...
plt.bar(x - w/2, values1, width=w)
plt.bar(x + w/2, values2, width=w)
plt.xticks(x, labels)
plt.bar(x, values2, bottom=values1)   # stacked
```

4.1 Grouped bars with np.arange() offsets

```
x = np.arange(len(poll))
plt.bar(x - 0.2, poll['boys_votes'], width=0.4, label='Boys')
plt.bar(x + 0.2, poll['girls_votes'], width=0.4, label='Girls')
```

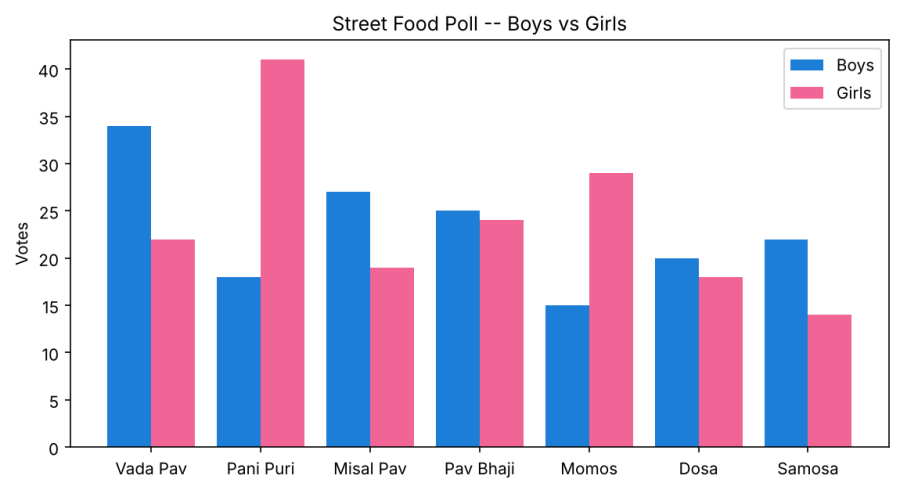


Explanation

np.arange(7) gives positions 0-6. Shifting boys 0.2 to the left and girls 0.2 to the right, each with width=0.4, places the two bars side by side without overlapping. The x-axis still shows numbers, not food names -- fixed next.

4.2 plt.xticks(x, labels), legend and title

```
x = np.arange(len(poll))
plt.figure(figsize=(9, 4.5))
plt.bar(x - 0.2, poll['boys_votes'], width=0.4, label='Boys',
        color='#1C7ED6')
plt.bar(x + 0.2, poll['girls_votes'], width=0.4, label='Girls',
        color='#F06595')
plt.xticks(x, poll['food'])
plt.title('Street Food Poll -- Boys vs Girls')
plt.ylabel('Votes')
plt.legend()
```

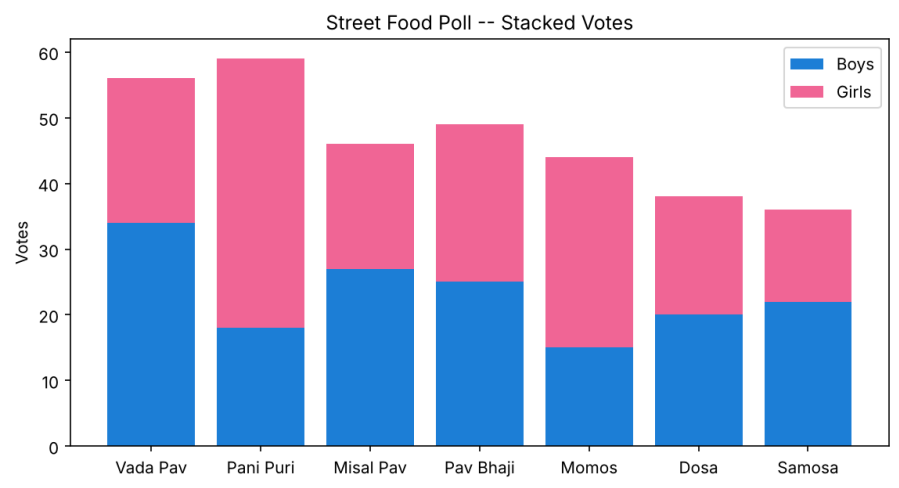


Explanation

plt.xticks(x, poll['food']) puts the food name under the centre of each pair. Now the comparison is instant: boys love Vada Pav, girls love Pani Puri and Momos, and Pav Bhaji is a tie.

4.3 Stacked bars with bottom=

```
plt.figure(figsize=(9, 4.5))
plt.bar(poll['food'], poll['boys_votes'], label='Boys',
        color='#1C7ED6')
plt.bar(poll['food'], poll['girls_votes'], label='Girls',
        color='#F06595', bottom=poll['boys_votes'])
plt.title('Street Food Poll -- Stacked Votes')
plt.ylabel('Votes')
plt.legend()
```

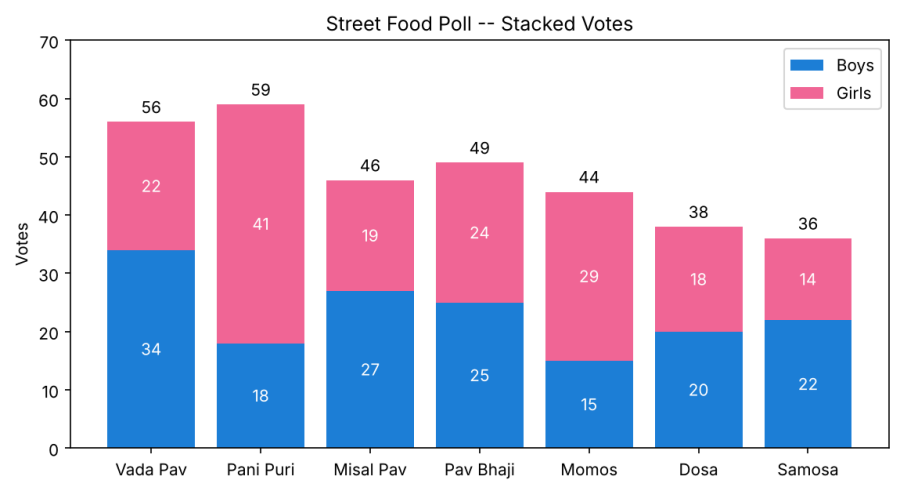


Explanation

No offsets this time: both bars use the same x. The trick is bottom= -- the girls' bar starts where the boys' bar ends instead of at zero. The total height of each column is the total votes.

4.4 Labels inside each stack segment

```
plt.figure(figsize=(9, 4.5))
b1 = plt.bar(poll['food'], poll['boys_votes'], label='Boys',
             color='#1C7ED6')
b2 = plt.bar(poll['food'], poll['girls_votes'], label='Girls',
             color='#F06595', bottom=poll['boys_votes'])
plt.bar_label(b1, label_type='center', color='white')
plt.bar_label(b2, label_type='center', color='white')
totals = (poll['boys_votes'] + poll['girls_votes']).astype(str)
plt.bar_label(b2, labels=totals, padding=3)
plt.title('Street Food Poll -- Stacked Votes')
plt.ylabel('Votes')
plt.legend()
plt.ylim(0, 70)
plt.show()
```



Explanation

label_type='center' writes each segment's own value in its middle (white text for contrast). A final bar_label() with labels=totals writes our own text -- the total votes -- on top of each column. Part and whole in one chart.

Teaching Note

Grouped = compare sub-groups with each other. Stacked = compare totals and see what they are made of. Ask students which of the two answers 'Did more girls or boys vote for Momos?' faster.

5. Histogram -- plt.hist()

A histogram is a **univariate** chart -- it needs only ONE numeric column. It groups the values into ranges called **bins** and draws a bar showing how many values fall in each bin. Use case: seeing the **distribution** -- where most values lie, and whether the data is symmetric or skewed. Our question: how long does a typical delivery take?

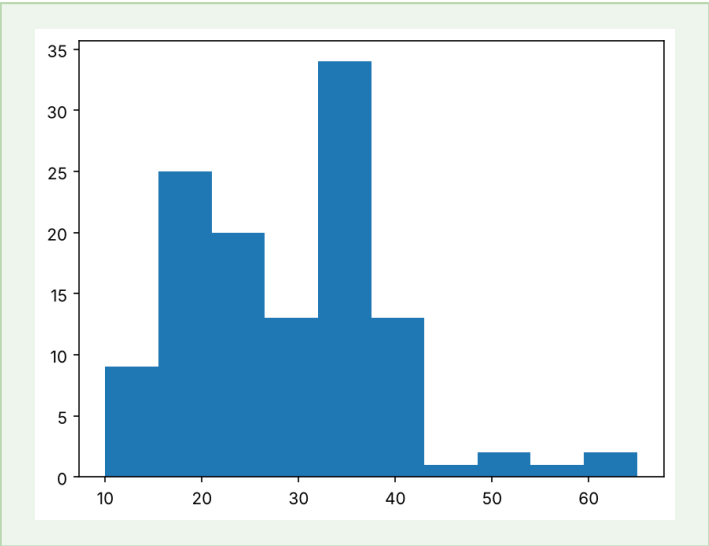
Dataset: food_delivery_orders.csv

Syntax

```
plt.hist(data, bins=, color=, edgecolor=, alpha=,
         label=, histtype=)
plt.axvline(x, color=, linestyle=, label=)
```

5.1 A plain histogram

```
plt.hist(food['delivery_time_min'])
```

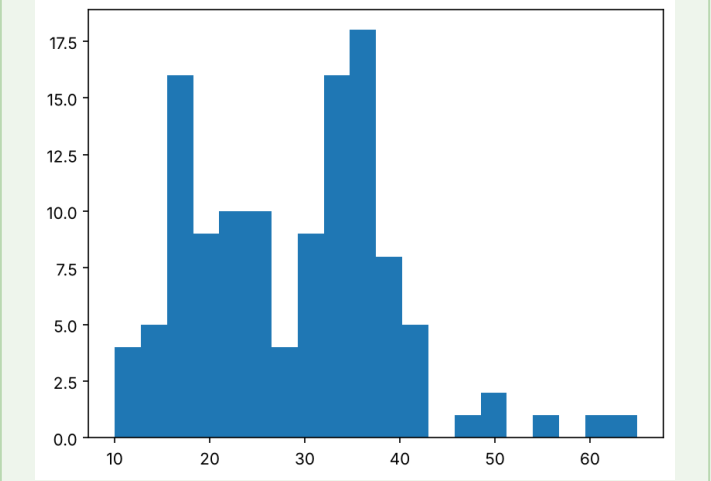


Explanation

With no bins= argument Matplotlib splits the full range (10 to 65 minutes) into 10 equal bins and counts the orders in each. The bar heights are COUNTS, not delivery times. Without borders, neighbouring bars blur together.

5.2 bins= as a number

```
plt.hist(food['delivery_time_min'], bins=20)
```

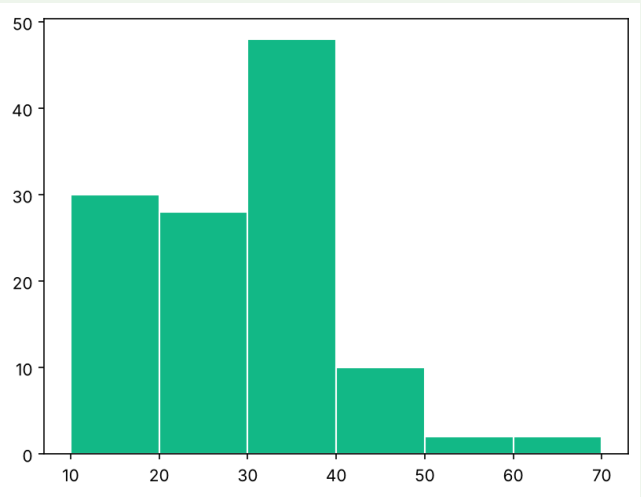


Explanation

bins=20 makes 20 narrower buckets. More bins show more detail but also more random noise; fewer bins smooth things out. There is no single 'right' number -- try 5, 10 and 20 and pick the one that shows the shape best.

5.3 bins= as a list of edges + edgecolor=

```
edges = [10, 20, 30, 40, 50, 60, 70]
plt.hist(food['delivery_time_min'], bins=edges,
         color='#12B886', edgecolor='white')
plt.xticks(edges)
```

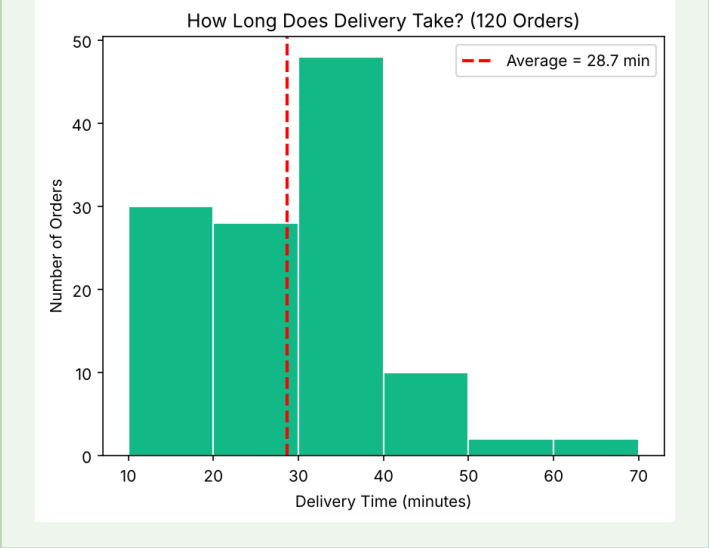


Explanation

A list gives the exact bin EDGES: [10, 20, ... 70] means 10-20, 20-30, ... 60-70 minutes -- easy, human-friendly ranges. edgecolor='white' separates the bars and plt.xticks(edges) puts a tick at every edge.

5.4 Labels and an average line with plt.axvline()

```
edges = [10, 20, 30, 40, 50, 60, 70]
avg = food['delivery_time_min'].mean()
plt.hist(food['delivery_time_min'], bins=edges,
         color='#12B886', edgecolor='white')
plt.axvline(avg, color='red', linestyle='--', linewidth=2,
            label='Average = ' + str(round(avg, 1)) + ' min')
plt.xticks(edges)
plt.title('How Long Does Delivery Take? (120 Orders)')
plt.xlabel('Delivery Time (minutes)')
plt.ylabel('Number of Orders')
plt.legend()
```

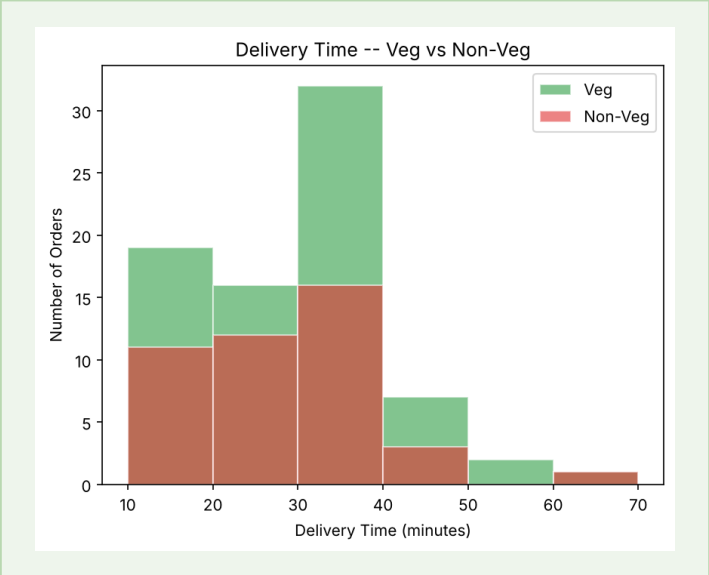


Explanation

plt.axvline(x) is the vertical twin of axhline(). Drawing it at the mean shows where the 'centre' of the distribution is. Most orders arrive within 40 minutes (the busiest bucket is 30-40 min), while a small tail of late orders stretches out to 60+ minutes.

5.5 Comparing two groups -- overlapping histograms

```
edges = [10, 20, 30, 40, 50, 60, 70]
veg = food[food['food_type'] == 'Veg']
nonveg = food[food['food_type'] == 'Non-Veg']
plt.hist(veg['delivery_time_min'], bins=edges, alpha=0.6,
         color='#2F9E44', edgecolor='white', label='Veg')
plt.hist(nonveg['delivery_time_min'], bins=edges, alpha=0.6,
         color='#E03131', edgecolor='white', label='Non-Veg')
plt.xticks(edges)
plt.title('Delivery Time -- Veg vs Non-Veg')
plt.xlabel('Delivery Time (minutes)')
plt.ylabel('Number of Orders')
plt.legend()
```

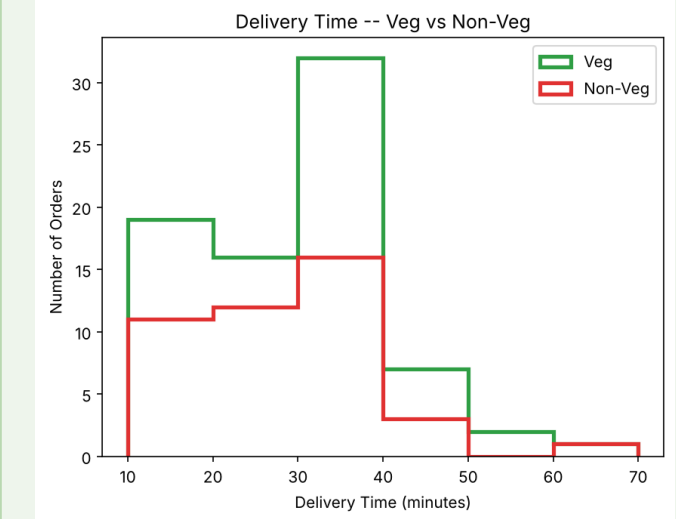


Explanation

Two plt.hist() calls on the same axes overlap. alpha=0.6 makes both see-through, so where they overlap you see a mixed colour. Using the SAME edges list for both is important -- otherwise the bars would not line up and the comparison would be unfair.

5.6 histtype='step' -- outline histograms

```
edges = [10, 20, 30, 40, 50, 60, 70]
veg = food[food['food_type'] == 'Veg']
nonveg = food[food['food_type'] == 'Non-Veg']
plt.hist(veg['delivery_time_min'], bins=edges, histtype='step',
         linewidth=2.5, color='#2F9E44', label='Veg')
plt.hist(nonveg['delivery_time_min'], bins=edges, histtype='step',
         linewidth=2.5, color='#E03131', label='Non-Veg')
plt.xticks(edges)
plt.title('Delivery Time -- Veg vs Non-Veg')
plt.xlabel('Delivery Time (minutes)')
plt.ylabel('Number of Orders')
plt.legend()
plt.show()
```



Explanation

histtype='step' draws only the outline of each histogram, with no fill. When comparing two or more groups, outlines never hide each other -- a cleaner alternative to transparency.

Teaching Note

Students often confuse histograms with bar charts. Bar chart = categories on the x-axis (Vada Pav, Dosa ...). Histogram = number RANGES on the x-axis (10-20 min, 20-30 min ...), with bars touching each other.

6. Box Plot -- plt.boxplot()

A box plot summarises a numeric column in five numbers: minimum, Q1 (25%), median (50%), Q3 (75%) and maximum -- and shows **outliers** as separate dots. Use case: comparing the **spread** of several groups side by side and spotting unusual values. Our question: which subject has the most consistent marks, and who are the outliers?

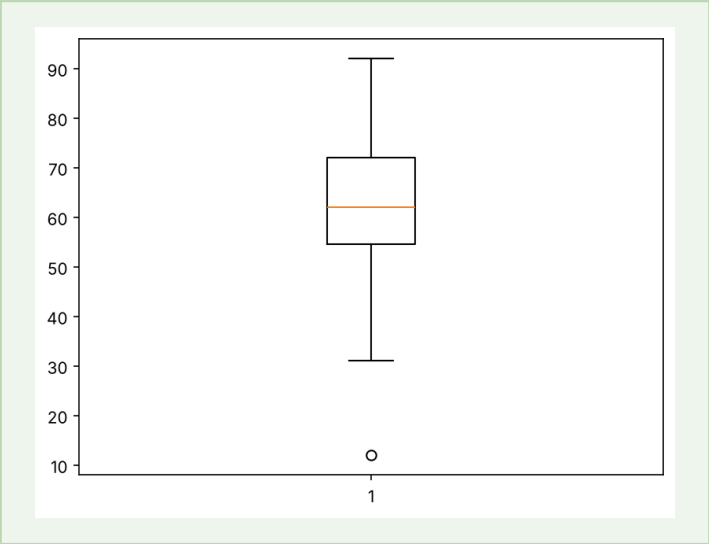
Dataset: student_exam_marks.csv

Syntax

```
plt.boxplot(data, tick_labels=, patch_artist=,
            showmeans=, orientation=)
box = plt.boxplot(...)   box['boxes']   # the box shapes
```

6.1 A plain box plot of one subject

```
plt.boxplot(marks['maths'])
```

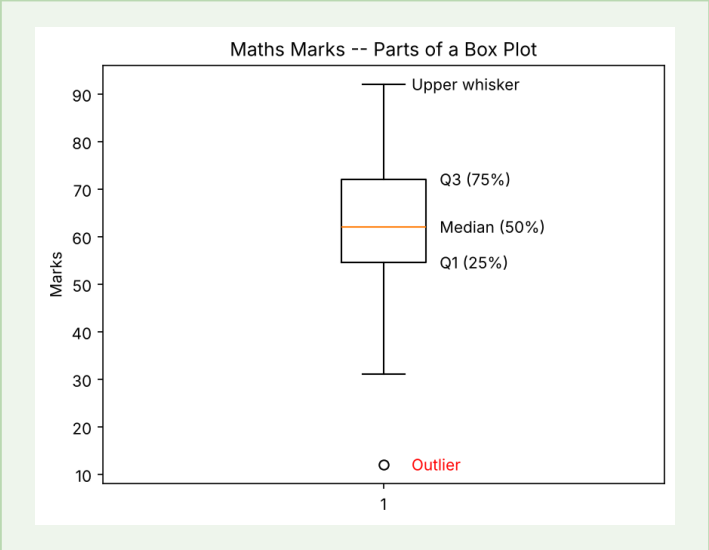


Explanation

One box for 30 Maths marks. The box covers the middle 50% of students, the orange line is the median, the whiskers reach the lowest and highest normal marks, and the lone circle at the bottom is an outlier (a mark of 12).

6.2 Reading a box plot -- labelling every part

```
q1 = marks['maths'].quantile(0.25)
median = marks['maths'].median()
q3 = marks['maths'].quantile(0.75)
top = marks['maths'].max()
plt.boxplot(marks['maths'])
plt.text(1.1, q1, 'Q1 (25%)', va='center')
plt.text(1.1, median, 'Median (50%)', va='center')
plt.text(1.1, q3, 'Q3 (75%)', va='center')
plt.text(1.05, top, 'Upper whisker', va='center')
plt.text(1.05, 12, 'Outlier', va='center', color='red')
plt.title('Maths Marks -- Parts of a Box Plot')
plt.ylabel('Marks')
```

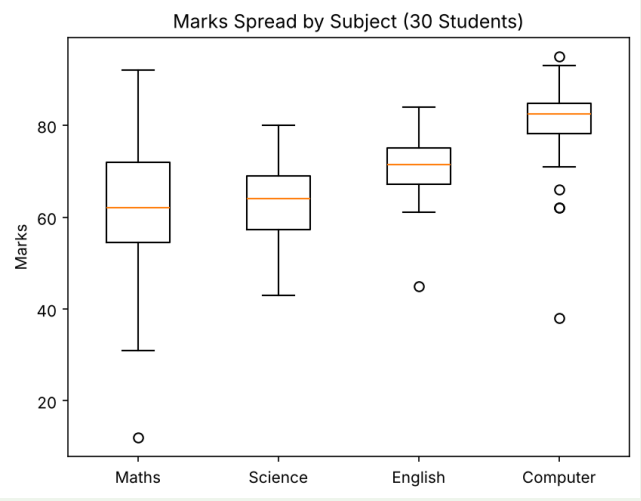


Explanation

plt.text(x, y, text) writes text anywhere on the chart. The box is drawn at x = 1, so x = 1.1 puts the text just to its right. Rule: any value further than 1.5 x (Q3 - Q1) beyond the box is treated as an outlier and drawn as a separate dot.

6.3 Several subjects side by side

```
subjects = ['maths', 'science', 'english', 'computer']
names = ['Maths', 'Science', 'English', 'Computer']
plt.boxplot(marks[subjects], tick_labels=names)
plt.title('Marks Spread by Subject (30 Students)')
plt.ylabel('Marks')
```

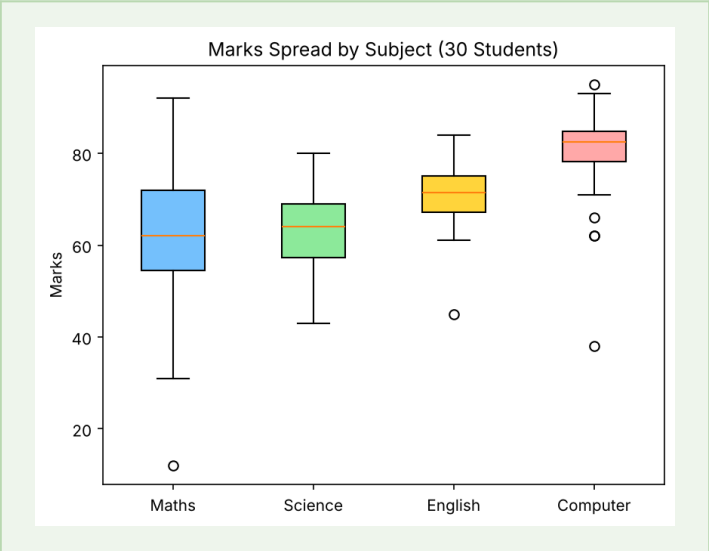


Explanation

Passing several columns draws one box per column; tick_labels= names them (older Matplotlib versions use labels= instead). Maths has the tallest box -- the widest spread -- while English is the most consistent. Computer has the highest median, but also a few students far below the rest (outliers).

6.4 patch_artist=True -- coloured boxes

```
subjects = ['maths', 'science', 'english', 'computer']
names = ['Maths', 'Science', 'English', 'Computer']
colours = ['#74C0FC', '#8CE99A', '#FFD43B', '#FFA8A8']
box = plt.boxplot(marks[subjects], tick_labels=names,
                  patch_artist=True)
for patch, colour in zip(box['boxes'], colours):
    patch.set_facecolor(colour)
plt.title('Marks Spread by Subject (30 Students)')
plt.ylabel('Marks')
```

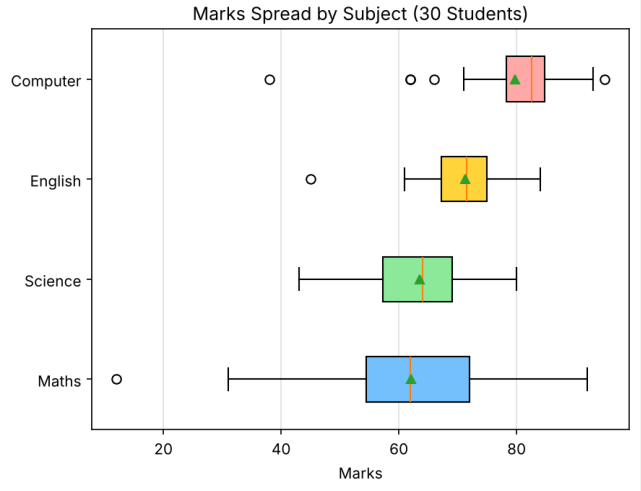


Explanation

patch_artist=True makes the boxes fillable shapes. plt.boxplot() returns a dictionary; box['boxes'] is the list of the 4 box shapes, and a simple for loop with zip() gives each one its own colour.

6.5 showmeans=True and a horizontal layout

```
subjects = ['maths', 'science', 'english', 'computer']
names = ['Maths', 'Science', 'English', 'Computer']
colours = ['#74C0FC', '#8CE99A', '#FFD43B', '#FFA8A8']
box = plt.boxplot(marks[subjects], tick_labels=names,
                  patch_artist=True, showmeans=True,
                  orientation='horizontal')
for patch, colour in zip(box['boxes'], colours):
    patch.set_facecolor(colour)
plt.title('Marks Spread by Subject (30 Students)')
plt.xlabel('Marks')
plt.grid(axis='x', alpha=0.4)
plt.show()
```



Explanation

showmeans=True adds a green triangle at the mean (average). Where the mean sits below the median (Maths), a few very low marks are pulling the average down. orientation='horizontal' lays the boxes sideways (older versions: vert=False), so the label moves to xlabel.

Teaching Note

Ask students to find the outlier students by name using pandas, e.g. marks[marks['english'] < 50] -- connecting the dot on the chart to a real row in the data makes outliers unforgettable.

7. Pie Chart -- plt.pie()

A pie chart shows how a **whole** is split into **parts**; each slice's angle is proportional to its share. Use it for a small number of categories (ideally up to 6). Use case: budget or market-share breakdowns. Our question: where did the family's money go in one full year?

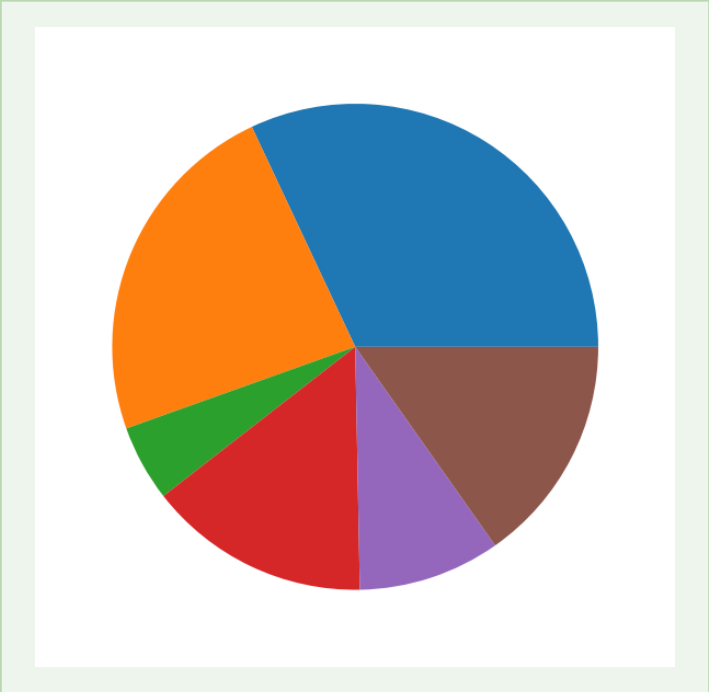
Dataset: monthly_household_expenses.csv

Syntax

```
plt.pie(values, labels=, autopct=, colors=, startangle=,
        explode=, shadow=, wedgeprops=, pctdistance=)
```

7.1 A plain pie chart of yearly spending

```
categories = ['rent', 'groceries', 'electricity',
              'travel', 'entertainment', 'shopping']
yearly = expenses[categories].sum()
plt.pie(yearly)
```

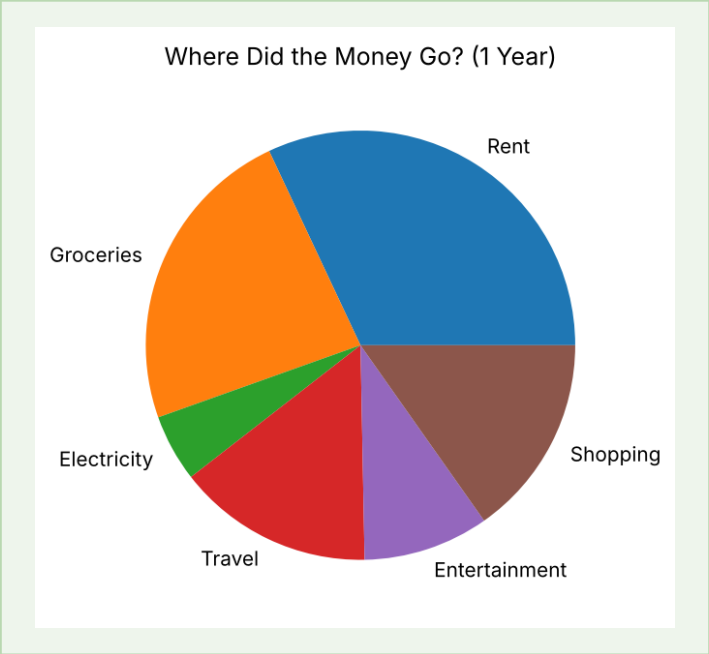


Explanation

expenses[categories].sum() adds up each column over 12 months, giving one yearly total per category. plt.pie() then turns those 6 numbers into 6 slices -- Matplotlib calculates the percentages itself. Without labels we cannot tell which slice is which.

7.2 labels=

```
categories = ['rent', 'groceries', 'electricity',
              'travel', 'entertainment', 'shopping']
yearly = expenses[categories].sum()
names = ['Rent', 'Groceries', 'Electricity',
         'Travel', 'Entertainment', 'Shopping']
plt.pie(yearly, labels=names)
plt.title('Where Did the Money Go? (1 Year)')
```

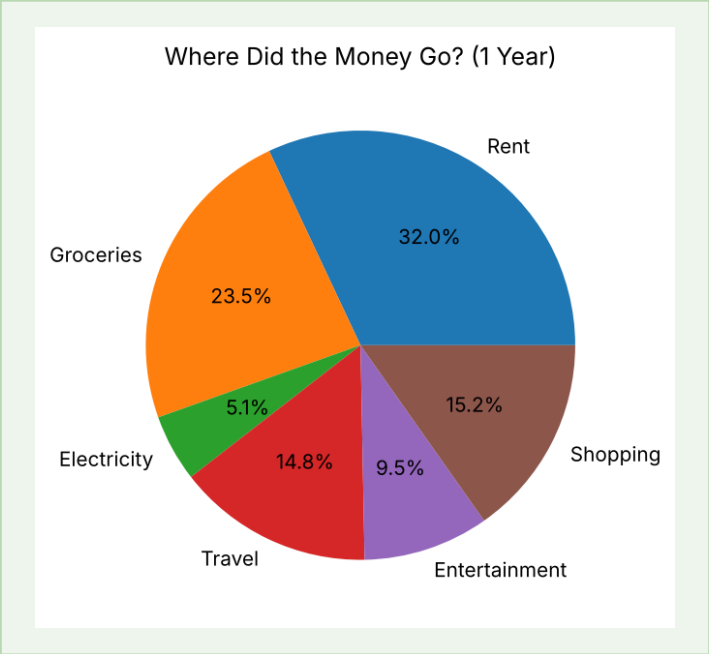


Explanation

labels= takes one name per slice, in the same order as the values. Now we can see rent is the biggest slice, followed by groceries.

7.3 autopct= -- percentages on the slices

```
categories = ['rent', 'groceries', 'electricity',
              'travel', 'entertainment', 'shopping']
yearly = expenses[categories].sum()
names = ['Rent', 'Groceries', 'Electricity',
         'Travel', 'Entertainment', 'Shopping']
plt.pie(yearly, labels=names, autopct='%1.1f%%')
plt.title('Where Did the Money Go? (1 Year)')
```



Explanation

autopct='%1.1f%%' prints each slice's percentage with 1 decimal place. Read it as: %1.1f = a number with one decimal, %% = a literal % sign.

7.4 colors= and startangle=

```
categories = ['rent', 'groceries', 'electricity',
              'travel', 'entertainment', 'shopping']
yearly = expenses[categories].sum()
names = ['Rent', 'Groceries', 'Electricity',
         'Travel', 'Entertainment', 'Shopping']
colours = ['#4C6EF5', '#40C057', '#FAB005',
           '#15AABF', '#BE4BDB', '#FA5252']
plt.pie(yearly, labels=names, autopct='%1.1f%%',
        colors=colours, startangle=90)
plt.title('Where Did the Money Go? (1 Year)')
```

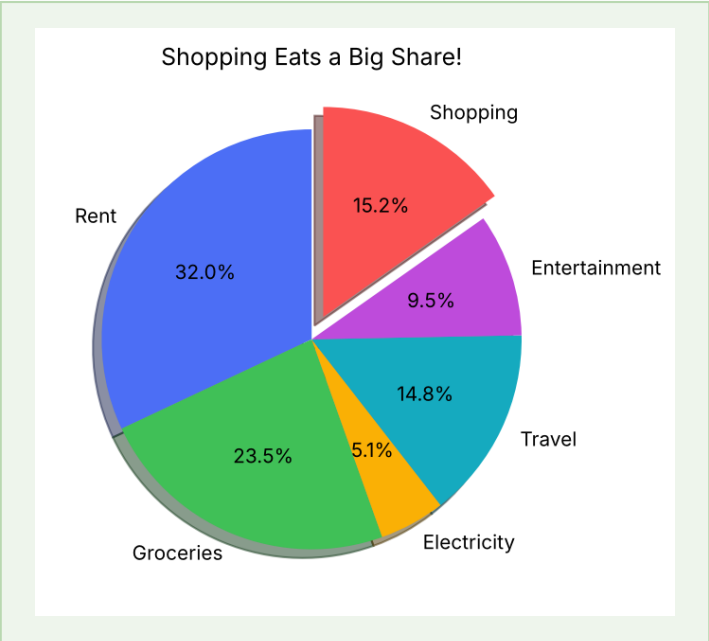


Explanation

colors= takes one colour per slice. startangle=90 starts the first slice (Rent) at the 12 o'clock position instead of 3 o'clock, which is how most people expect a pie to begin.

7.5 explode= and shadow=

```
categories = ['rent', 'groceries', 'electricity',
              'travel', 'entertainment', 'shopping']
yearly = expenses[categories].sum()
names = ['Rent', 'Groceries', 'Electricity',
         'Travel', 'Entertainment', 'Shopping']
colours = ['#4C6EF5', '#40C057', '#FAB005',
           '#15AABF', '#BE4BDB', '#FA5252']
plt.pie(yearly, labels=names, autopct='%1.1f%%',
        colors=colours, startangle=90,
        explode=[0, 0, 0, 0, 0, 0.12], shadow=True)
plt.title('Shopping Eats a Big Share!')
```

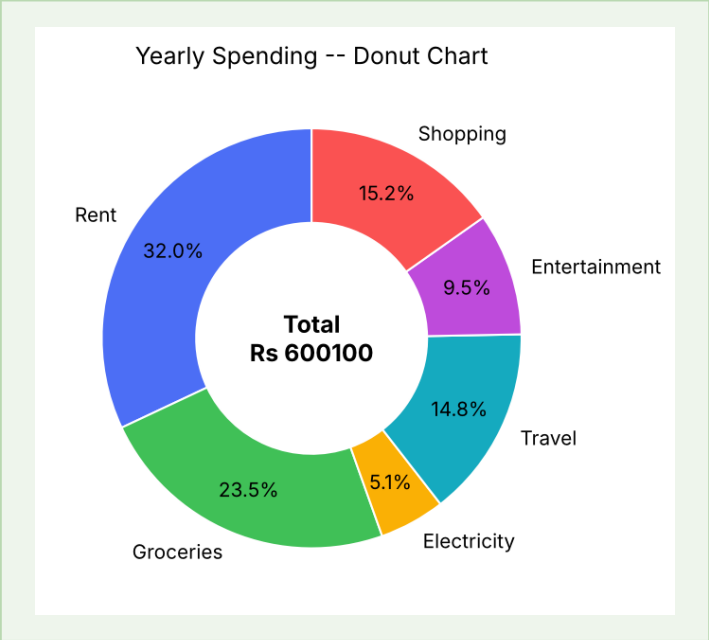


Explanation

explode= takes one offset per slice; 0 keeps a slice in place and 0.12 pulls Shopping slightly out to highlight it (thanks to the big Diwali shopping in October). shadow=True adds a drop-shadow.

7.6 A donut chart with wedgeprops=

```
categories = ['rent', 'groceries', 'electricity',
              'travel', 'entertainment', 'shopping']
yearly = expenses[categories].sum()
names = ['Rent', 'Groceries', 'Electricity',
         'Travel', 'Entertainment', 'Shopping']
colours = ['#4C6EF5', '#40C057', '#FAB005',
           '#15AABF', '#BE4BDB', '#FA5252']
total = yearly.sum()
plt.pie(yearly, labels=names, autopct='%1.1f%%',
        colors=colours, startangle=90, pctdistance=0.78,
        wedgeprops=dict(width=0.45, edgecolor='white'))
plt.text(0, 0, 'Total\nRs ' + str(total), ha='center',
         va='center', fontsize=12, fontweight='bold')
plt.title('Yearly Spending -- Donut Chart')
plt.show()
```



Explanation

wedgeprops=dict(width=0.45) keeps only the outer 45% of each slice, cutting a hole in the middle -- a **donut chart**. pctdistance=0.78 moves the percentages onto the ring, and plt.text(0, 0, ...) writes the yearly total in the empty centre ('\n' starts a new line).

Teaching Note

Pie charts become unreadable beyond about 6 slices, and humans judge angles poorly. Show the same yearly data as a sorted plt.barh() and ask students which chart makes 'Travel vs Groceries' easier to compare.

8. Subplots -- plt.subplot() / plt.subplots()

Subplots place **several charts in one figure**, like panels in a dashboard or a report. This is also where we move from the simple plt.xxx() style to the **object-oriented style**: fig is the whole canvas, ax (axes) is one chart inside it. Use case: comparing related views of the same data side by side -- here, Pune's yearly weather.

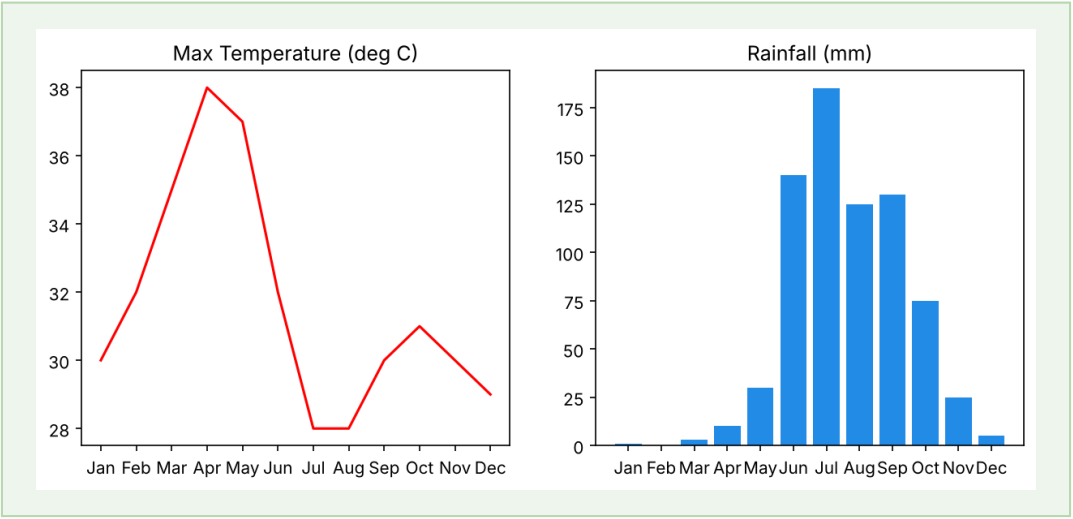
Dataset: pune_monthly_weather.csv

Syntax

```
plt.subplot(rows, cols, index)
fig, ax = plt.subplots(figsize=)
fig, axes = plt.subplots(rows, cols, figsize=)
ax.plot() ax.set_title() ax.set_xlabel() ax.set_ylabel()
fig.suptitle()   plt.tight_layout()   ax2 = ax.twinx()
```

8.1 plt.subplot() -- two charts side by side

```
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(weather['month'], weather['max_temp_c'], color='red')
plt.title('Max Temperature (deg C)')
plt.subplot(1, 2, 2)
plt.bar(weather['month'], weather['rainfall_mm'], color='#228BE6')
plt.title('Rainfall (mm)')
```

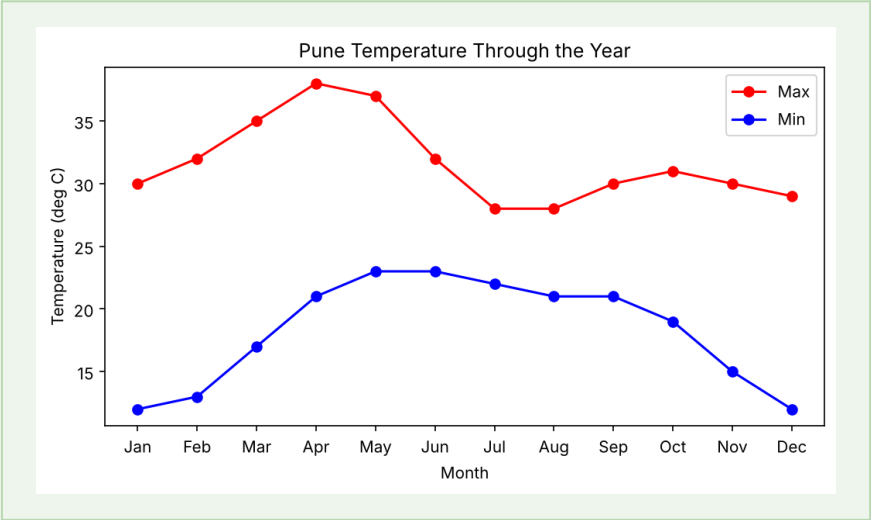


Explanation

plt.subplot(rows, cols, index) splits the figure into a grid and makes panel number 'index' active. (1, 2, 1) = 1 row, 2 columns, first panel; everything after it draws there until plt.subplot(1, 2, 2) switches to the second panel.

8.2 fig, ax = plt.subplots() -- the object-oriented style

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(weather['month'], weather['max_temp_c'], color='red',
        marker='o', label='Max')
ax.plot(weather['month'], weather['min_temp_c'], color='blue',
        marker='o', label='Min')
ax.set_title('Pune Temperature Through the Year')
ax.set_xlabel('Month')
ax.set_ylabel('Temperature (deg C)')
ax.legend()
```

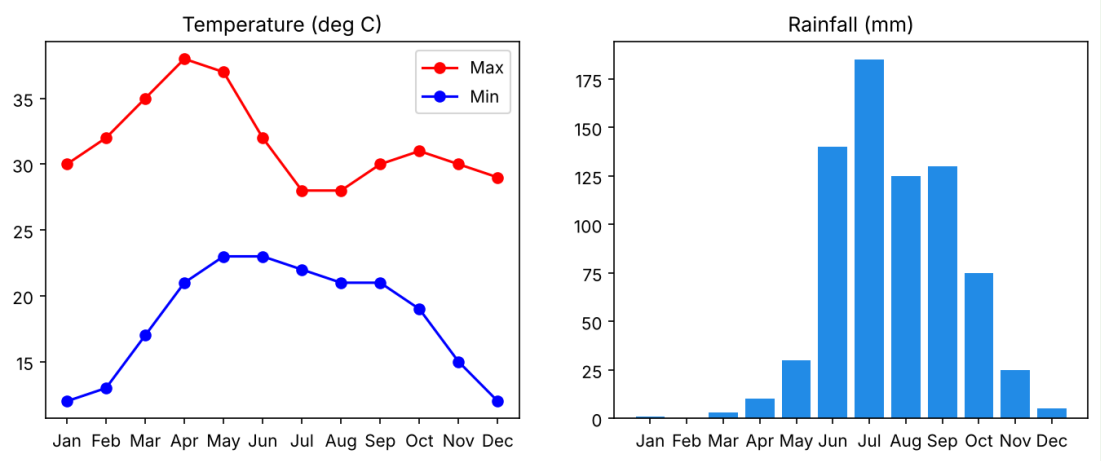


Explanation

plt.subplots() returns two objects: fig (the canvas) and ax (the chart area). We now call methods ON ax. Most names are the same as before, but labels get a set_ prefix: plt.title() becomes ax.set_title(), plt.xlabel() becomes ax.set_xlabel(), plt.ylim() becomes ax.set_ylim().

8.3 fig, axes = plt.subplots(1, 2) -- an array of axes

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(weather['month'], weather['max_temp_c'],
             color='red', marker='o', label='Max')
axes[0].plot(weather['month'], weather['min_temp_c'],
             color='blue', marker='o', label='Min')
axes[0].set_title('Temperature (deg C)')
axes[0].legend()
axes[1].bar(weather['month'], weather['rainfall_mm'],
            color='#228BE6')
axes[1].set_title('Rainfall (mm)')
```

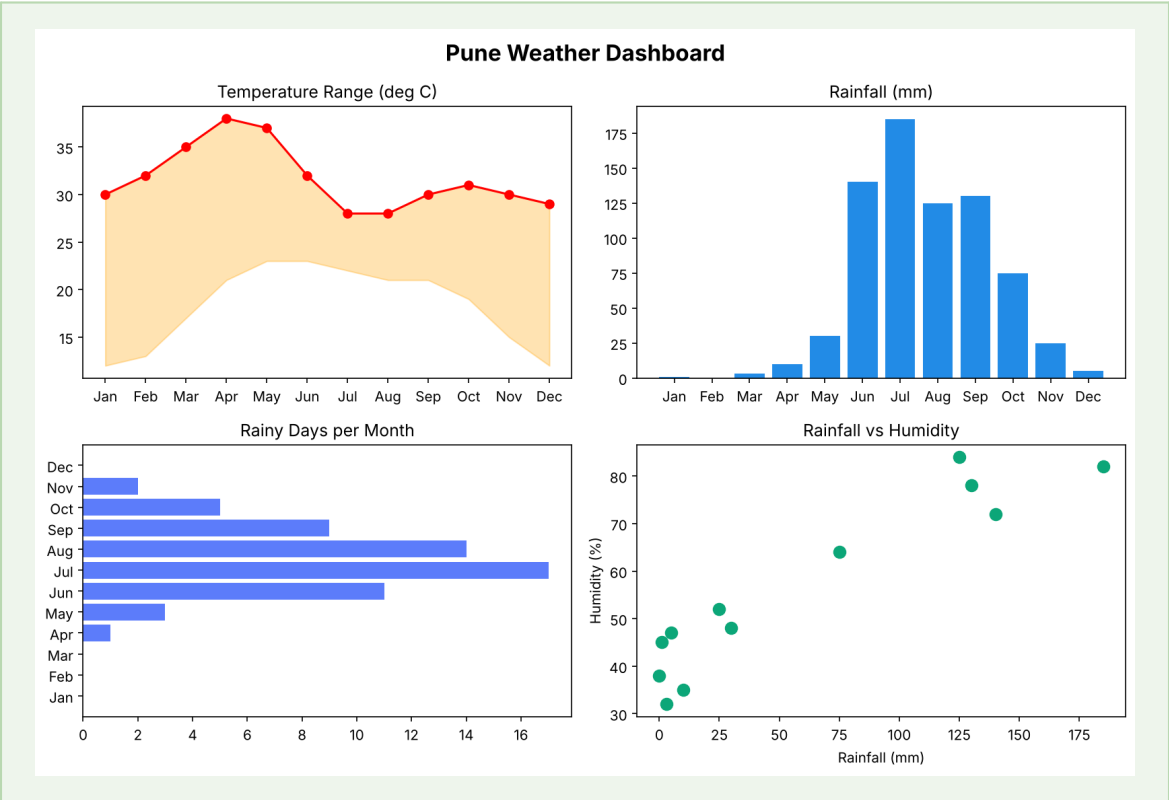


Explanation

With 1 row and 2 columns, axes is a NumPy array of 2 axes objects: axes[0] is the left chart, axes[1] the right one. There is no 'switching' as in plt.subplot() -- you simply say which ax to draw on.

8.4 A 2 x 2 weather dashboard with suptitle() and tight_layout()

```
fig, axes = plt.subplots(2, 2, figsize=(11, 7.5))
axes[0, 0].plot(weather['month'], weather['max_temp_c'],
                color='red', marker='o')
axes[0, 0].fill_between(weather['month'], weather['min_temp_c'],
                        weather['max_temp_c'], color='orange',
                        alpha=0.3)
axes[0, 0].set_title('Temperature Range (deg C)')
axes[0, 1].bar(weather['month'], weather['rainfall_mm'],
               color='#228BE6')
axes[0, 1].set_title('Rainfall (mm)')
axes[1, 0].barh(weather['month'], weather['rainy_days'],
                color='#5C7CFA')
axes[1, 0].set_title('Rainy Days per Month')
axes[1, 1].scatter(weather['rainfall_mm'],
                   weather['humidity_pct'], color='#0CA678', s=70)
axes[1, 1].set_title('Rainfall vs Humidity')
axes[1, 1].set_xlabel('Rainfall (mm)')
axes[1, 1].set_ylabel('Humidity (%)')
fig.suptitle('Pune Weather Dashboard', fontsize=16,
             fontweight='bold')
plt.tight_layout()
```

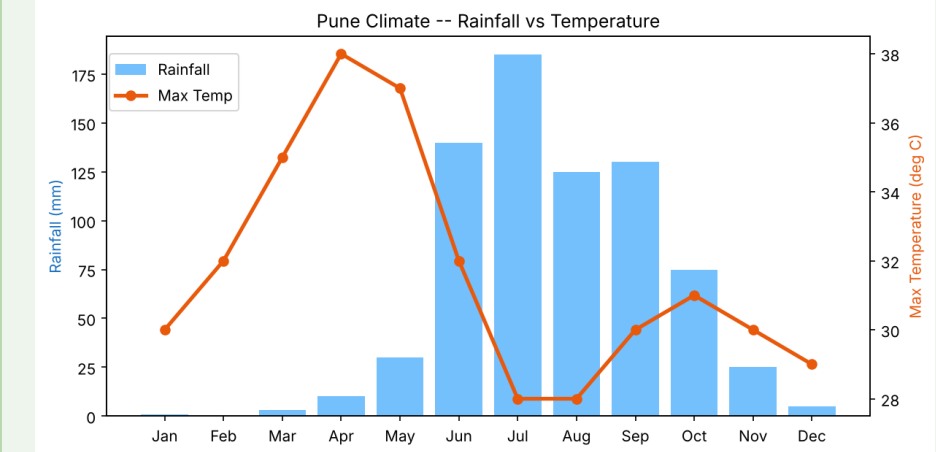


Explanation

With 2 rows and 2 columns, axes becomes a 2-D grid: axes[row, col]. Each panel can hold a different chart type -- line + area, bar, horizontal bar and scatter -- all learned in earlier topics. fig.suptitle() puts one big title over the whole figure and plt.tight_layout() auto-adjusts the spacing so titles and labels never overlap.

8.5 Two y-axes on one chart with ax.twinx()

```
fig, ax1 = plt.subplots(figsize=(9, 4.5))
ax1.bar(weather['month'], weather['rainfall_mm'],
        color='#74C0FC', label='Rainfall')
ax1.set_ylabel('Rainfall (mm)', color='#1971C2')
ax2 = ax1.twinx()
ax2.plot(weather['month'], weather['max_temp_c'], color='#E8590C',
         marker='o', linewidth=2.5, label='Max Temp')
ax2.set_ylabel('Max Temperature (deg C)', color='#E8590C')
ax1.set_title('Pune Climate -- Rainfall vs Temperature')
fig.legend(loc='upper left', bbox_to_anchor=(0.12, 0.86))
plt.show()
```



Explanation

Rainfall (0-185 mm) and temperature (28-38 deg C) have very different scales. ax1.twinx() creates a second axes sharing the same x-axis but with its own y-axis on the right. This is the classic 'climate chart': as the monsoon rain rises in June-July, the temperature drops.

Teaching Note

Teach the mapping table on the board: plt.title -> ax.set_title, plt.xlabel -> ax.set_xlabel, plt.xlim -> ax.set_xlim, plt.xticks -> ax.set_xticks. Everything else (plot, bar, scatter, hist, pie, legend) keeps the same name.